Code : LGLSI-3-26-7

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Monastir
Faculté des Sciences de Monastir

Rapport de Stage de Fin d'Etudes
pour l'obtention de la

Licence Génie Logiciel et Système d'Information

Sujet : **Musique générative et architecture microservices**

Élaboré par : **KHLIF MOHAMED YASSINE**

Soutenu le 12/06/2026 à 09:30 à la salle Amphi 1 devant le Jury :

Président :	**Mohsen Maraoui**
Rapporteur :	**Yousri Daldoul**
Encadrant Universitaire :	**Mounir Zrigui**
Encadrant Professionnel :	**Mounir Zrigui**

Effectué à : **Laboratoire RLANTIS-FSM**

Année Universitaire 2025/2026


OPEN GENERATIVE
STUDIOS.

Remerciements

Au terme de ce travail, il m'est agréable de remercier toutes les personnes qui ont contribué à sa réalisation. Je commence naturellement par ma famille, qui a supporté mes absences, mes doutes et mes longues nuits de travail avec une patience remarquable. Je pense tout particulièrement à mon père, Noureddine Khlif, ma mère, Wafa Rebai, ma sœur, Tasnim Khlif, et mon frère, Ayoub Khlif, ainsi qu'à l'ensemble de ma famille élargie. Leur soutien silencieux mais constant a été mon point d'ancrage tout au long de ce projet.

Je remercie également mes amis pour leurs encouragements sincères et pour avoir su, souvent sans le savoir, me redonner de l'élan quand j'en avais le plus besoin.

Mes remerciements vont aussi à l'ensemble des enseignants de la Faculté des Sciences de l'Université de Monastir, dont les enseignements se retrouvent, d'une manière ou d'une autre, sur chaque page de ce mémoire. Je tiens à exprimer ma profonde gratitude à mon professeur Mounir Zrigui, qui a cru en moi et dans le potentiel de cette idée dès le début. Je remercie également les enseignants de l'Académie Militaire de Fondouk Jdid, qui ont façonné ma façon de penser et de travailler.

Ce projet est né à l'intersection de deux mondes qui me définissent : la musique et l'ingénierie logicielle. Je ne peux donc pas ne pas mentionner mes professeurs de musique, ceux qui m'ont accompagné depuis l'enfance. Je pense en premier lieu à mon professeur Mourad Bougares, qui a posé en moi les premières pierres d'une passion qui n'a cessé de grandir. Il a cru en mon potentiel et sa confiance m'a donné l'envie de ne pas m'arrêter là et l'ambition de faire de ce travail les premières fondations d'une future startup.

Enfin, je remercie la communauté open source, dont la générosité et l'esprit de partage ont rendu ce projet techniquement possible. Par leur engagement collectif et leur volonté de rendre le savoir accessible à tous, ils contribuent chaque jour à façonner un secteur technologique plus ouvert, plus accessible et porteur d'un avenir meilleur pour l'humanité tout entière.

Résumé

Les plateformes de génération musicale dominantes produisent de l'audio de manière opaque, sans contrôle structurel ni artefacts intermédiaires inspectables. Ce rapport présente "Open Generative Studios", une plateforme web de génération musicale orchestrale conçue comme une alternative transparente, reproductible et déployable sans GPU.

L'architecture retenue est une Staged Event-Driven Architecture (SEDA) à six microservices connectés par des files Redis : application web cliente, moteur de composition, service MIDI, service de rendu audio, service de notation et passerelle de notification. Le développement a suivi le Processus Unifié en quatre phases : Création, Élaboration, Construction et Transition.

La génération musicale repose sur une pile hybride : un grand modèle de langage (Claude Haiku) traduit le prompt utilisateur en document d'intention structuré (JSON), puis trois modèles légers prennent le relais. Un Random Forest (200 arbres, corpus MAESTRO) prédit la vélocité et le delta de durée de chaque note. Une chaîne de Markov d'ordre 3 (corpus Lakh MIDI) modélise la progression mélodique par genre. Un K-Means (corpus Lakh MIDI) sélectionne les patterns rythmiques selon le niveau d'énergie de chaque section. Une chaîne déterministe de post-traitement garantit ensuite la cohérence musicale (filtrage tonal, contrainte de tessiture, normalisation de la timeline), avant un rendu audio entièrement scriptable via Sfizz/SFZ. Le pipeline produit à chaque étape des artefacts inspectables et rejouables : couches MIDI, stems WAV, mix final et partition PDF.

Le système intègre par ailleurs un module de sécurité complet (authentification JWT, OAuth, chiffrement, limitation de débit), une facturation par crédits (Lemon Squeezy), une observabilité en temps réel via Server-Sent Events et une gestion des erreurs via Dead Letter Queues, un pipeline CI/CD automatisé via GitHub Actions, et un déploiement multi-cloud reproductible sur DigitalOcean Droplets et Azure App Service, avec Supabase et MongoDB Atlas pour la persistance des données.

Table des matières

Remerciements i

Résumé iii

Table des matières v

Table des figures ix

Liste des tableaux xi

Glossaire xiii

Introduction générale 1

1 Création : Vision, contexte et socle technologique 5

1.1 Introduction . . . 5
1.2 Analyse de l'existant . . . 5
1.3 Vue d'ensemble du système . . . 6
1.4 Justification des choix technologiques et architecturaux . . . 8
1.4.1 Interface et applications web . . . 8
1.4.2 Persistance et communication inter-services . . . 8
1.4.3 Intelligence artificielle et machine learning . . . 8
1.4.4 Traitement musical et audio . . . 9
1.4.5 Sécurité . . . 9
1.4.6 Infrastructure et déploiement . . . 9
1.4.7 Facturation et communication . . . 9
1.4.8 Outillage de documentation . . . 10
1.5 Architecture cible : pipeline à 6 microservices . . . 10
1.5.1 Pourquoi une architecture microservices . . . 10
1.5.2 Pourquoi SEDA . . . 10
1.5.3 Structure logique du système . . . 11

1.5.4 Les six services du pipeline . . . 11
1.6 Mise en place de l'infrastructure initiale . . . 13
1.6.1 Environnement Docker Compose . . . 13
1.6.2 Validation initiale de la communication par files . . . 13
1.7 Bilan de la Création . . . 13
1.7.1 Résultats atteints . . . 13
1.7.2 Limitations identifiées . . . 13
1.7.3 Défis reportés vers l'Élaboration . . . 14
1.8 Conclusion . . . 14

2 Élaboration : Architecture et pipeline de génération musicale 15
2.1 Introduction . . . 15
2.2 Architecture du système . . . 15
2.2.1 Mise en œuvre SEDA . . . 15
2.2.2 Contrats de files d'attente et résilience . . . 15
2.2.3 Flux bout en bout . . . 16
2.3 Service de passerelle applicative . . . 17
2.4 Service de composition . . . 18
2.4.1 Extraction d'intention et résolution instrumentale . . . 18
2.4.2 Pile de génération musicale : une approche hybride . . . 18
2.4.3 Articulation des modèles dans le flux de composition . . . 24
2.4.4 Persistance et transmission à l'étape suivante . . . 25
2.5 Service de post-traitement MIDI . . . 25
2.6 Service de rendu audio . . . 25
2.7 Service de notation musicale . . . 26
2.8 Service de notification . . . 27
2.9 Bilan de l'Élaboration . . . 27
2.9.1 Résultats atteints . . . 27
2.9.2 Limitations identifiées . . . 27
2.9.3 Défis reportés vers la Construction . . . 27
2.10 Conclusion . . . 27

3 Construction : Services applicatifs, expérience utilisateur et qualité 29
3.1 Introduction . . . 29
3.2 Sécurité et authentification . . . 29
3.2.1 Mécanismes principaux . . . 29
3.2.2 Flux d'authentification . . . 30
3.3 Facturation et gestion des bundles . . . 31
3.3.1 Intégration Lemon Squeezy . . . 31
3.3.2 Bundles de clés . . . 32

3.3.3 Traitement des webhooks . . . 33
3.3.4 Limites d'utilisation et coût dynamique . . . 33
3.4 Application web cliente . . . 33
3.4.1 Parcours public . . . 33
3.4.2 Authentification . . . 34
3.4.3 Studio conversationnel . . . 34
3.4.4 Tableau de bord client . . . 35
3.4.5 Compte et bundles . . . 35
3.5 Tableau de bord administrateur . . . 36
3.5.1 Objectif . . . 36
3.5.2 Administration des utilisateurs et des bundles . . . 36
3.5.3 Métriques et analytique . . . 37
3.6 Assurance qualité et observabilité . . . 39
3.6.1 Positionnement opérationnel de la pile de composition . . . 39
3.6.2 Conception de la couche déterministe . . . 39
3.6.3 Profils d'efficacité des composantes . . . 41
3.6.4 Validation au niveau des services . . . 41
3.6.5 Vérification de l'intégration entre les étapes de files d'attente . . . 41
3.6.6 Tests de scénarios de bout en bout . . . 42
3.6.7 Surveillance opérationnelle . . . 42
3.7 Bilan de la Construction . . . 42
3.7.1 Résultats atteints . . . 42
3.7.2 Limitations identifiées . . . 42
3.7.3 Défis reportés vers la Transition . . . 42
3.8 Conclusion . . . 43

4 Transition : Déploiement, exploitation et perspectives 45
4.1 Introduction . . . 45
4.2 Infrastructure et déploiement . . . 45
4.2.1 Environnement local . . . 45
4.2.2 Architecture de production . . . 45
4.2.3 Terminaison TLS et reverse proxy . . . 47
4.2.4 Domaine et configuration DNS . . . 47
4.2.5 Infrastructure de messagerie transactionnelle . . . 48
4.2.6 Sécurité et déploiement continu . . . 48
4.2.7 Analyse des coûts . . . 49
4.3 Leçons apprises . . . 50
4.3.1 Décisions architecturales . . . 50
4.3.2 Trajectoire d'expérimentation Machine Learning . . . 50

4.3.3 Apprentissages en systèmes distribués 50
4.4 Bilan de la Transition 51
4.4.1 Résultats atteints 51
4.4.2 Limitations identifiées 51
4.4.3 Perspectives 52
4.5 Conclusion 52

Conclusion générale **53**

Annexe A : Référentiel technologique et audio **55**

Annexe B : Catalogue VPO **57**

Annexe C : Dictionnaire musical détaillé **59**

Annexe D : Exemple de partition : Ode à la joie de Beethoven **61**

Annexe E : Rôle et structure du document d'intention musicale **63**

Annexe F : Confirmation de paiement et crédit des clés via webhook **65**

Annexe G : Référence des endpoints API **67**

Bibliographie **69**

Table des figures

1.1 Diagramme de cas d'utilisation : parcours utilisateur et services principaux 7
1.2 Diagramme de classes : structure logique du système et relations clés 11

2.1 Représentation de l'architecture SEDA et interactions entre les services 16
2.2 Diagramme de séquence : génération, envoi du premier message et mise en file de composition 17
2.3 Diagramme de séquence : interruption, annulation d'une génération en cours 18
2.4 Vue consolidée du pipeline de génération musicale 26

3.1 Diagramme de séquence : processus d'inscription et de vérification d'adresse e-mail 30
3.2 Diagramme de séquence : processus de connexion et de déconnexion utilisateur . . 31
3.3 Diagramme de séquence : facturation, création de la session de paiement (checkout) 32
3.4 Interface d'accueil de la plateforme. 33
3.5 Tiroir d'inscription et de connexion (email ou Google). 34
3.6 Studio conversationnel : prompt, génération et téléchargement des artefacts. 34
3.7 Tableau de bord client : historique des compositions et artefacts. 35
3.8 Compte client : consommation de clés et gestion des bundles. 35
3.9 Tableau de bord administrateur : indicateurs globaux, utilisateurs et revenus. 36
3.10 Administration : liste des utilisateurs, soldes de clés et historique d'achat. 36
3.11 Administration : configuration des bundles Prelude, Concerto et Mozart. 37
3.12 Administration : configuration du coût de génération et tarification dynamique. . . 37
3.13 Diagramme de séquence : administration, gestion des plans et configuration des coûts 38

4.1 Architecture de déploiement multi-cloud en production 46
B.1 Orchestre (visuel d'illustration du VPO) 57
D.1 Exemple de partition : Ode à la joie de Beethoven 61
F.1 Diagramme de séquence : facturation, confirmation et crédit des clés via webhook . 65

Liste des tableaux

TABLEAU 2.1 – Données et séparation train/test pour la composante expressive 19
TABLEAU 2.2 – Vecteur de caractéristiques de la composante expressive (21 dimensions) 20
TABLEAU 2.3 – Métriques de validation de la composante expressive 21
TABLEAU 2.4 – Données et séparation train/test pour la composante mélodique 22
TABLEAU 2.5 – Interprétation de la perplexité par intervalle 22
TABLEAU 2.6 – Métriques par genre pour la composante mélodique (ensemble de test) 23
TABLEAU 2.7 – Données et séparation train/test pour la composante rythmique 23
TABLEAU 2.8 – Métriques par genre pour la composante rythmique (train et test) . . . 24

TABLEAU 3.1 – Bundles de clés 32

TABLEAU 4.1 – Enregistrements DNS principaux 47
TABLEAU 4.2 – Composantes du système, fournisseurs et coûts 49
TABLEAU A.1 – Technologies utilisées 55
TABLEAU C.1 – Dictionnaire des termes musicaux 59
TABLEAU G.1 – Endpoints API client et administrateur 67

Glossaire

ACME Automated Certificate Management Environment : protocole standard permettant l'automatisation complète de l'émission, de la validation et du renouvellement des certificats TLS.

Amazon SES Amazon Simple Email Service : service cloud d'Amazon dédié à l'envoi d'e-mails transactionnels et marketing à grande échelle.

API Application Programming Interface : interface logicielle permettant à des applications d'accéder à des fonctionnalités ou à des données de manière structurée.

Artefact Résultat généré par un traitement ou un pipeline, sous forme de fichier ou de donnée exploitable.

Bagging Bootstrap Aggregating : méthode d'ensemble entraînant plusieurs modèles sur des sous-échantillons aléatoires afin de réduire la variance.

BPM Beats Per Minute : unité mesurant le tempo musical en nombre de battements par minute.

CDN Content Delivery Network : infrastructure distribuée permettant de diffuser du contenu web au plus près des utilisateurs.

CI/CD Continuous Integration / Continuous Delivery : ensemble de pratiques automatisant l'intégration, les tests et le déploiement logiciel.

Conteneur Environnement d'exécution isolé et portable permettant d'exécuter une application avec ses dépendances.

Corpus Ensemble structuré de données utilisé pour l'entraînement ou l'évaluation de modèles.

CPU Central Processing Unit : processeur principal exécutant les instructions d'un système informatique.

CSR Client-Side Rendering : technique de rendu d'interface exécutée directement dans le navigateur.

DAW Digital Audio Workstation : logiciel dédié à la production, l'enregistrement et l'édition audio.

DKIM DomainKeys Identified Mail : mécanisme de signature cryptographique garantissant l'authenticité des e-mails.

DLQ Dead Letter Queue : file d'attente recevant les messages en échec de traitement.

DMARC Domain-based Message Authentication, Reporting and Conformance : politique de validation des e-mails combinant SPF et DKIM.

DNS Domain Name System : système traduisant les noms de domaine en adresses IP.

Feature Variable d'entrée utilisée par un modèle d'apprentissage automatique.

File de messages Mécanisme de communication asynchrone basé sur des files d'attente de messages.

GPU Graphics Processing Unit : processeur spécialisé dans le calcul parallèle intensif.

Headless Mode d'exécution sans interface graphique.

HMAC Hash-based Message Authentication Code : mécanisme d'authentification basé sur une fonction de hachage et une clé secrète.

HPF High-Pass Filter : filtre audio laissant passer les hautes fréquences tout en atténuant les basses.

Idempotence Propriété d'une opération produisant toujours le même résultat, quel que soit le nombre de fois où elle est exécutée.

JSON JavaScript Object Notation : format léger de structuration et d'échange de données.

JWT JSON Web Token : jeton signé utilisé pour l'authentification et le transfert sécurisé d'informations.

LLM Large Language Model : modèle d'intelligence artificielle entraîné sur de vastes corpus de texte.

Microservice Architecture logicielle composée de services indépendants et spécialisés.

MIDI Musical Instrument Digital Interface : protocole de communication entre instruments et logiciels musicaux.

MVP Minimum Viable Product : version initiale minimale d'un produit permettant de valider une idée.

PaaS Platform as a Service : modèle cloud fournissant une plateforme complète de déploiement applicatif.

PDF Portable Document Format : format de document universel conservant la mise en page.

Pipeline Chaîne de traitements exécutés de manière séquentielle ou structurée.

Prompt Instruction textuelle fournie à un modèle pour générer une réponse.

SaaS Software as a Service : modèle de distribution logicielle accessible via Internet.

SEDA Staged Event-Driven Architecture : architecture logicielle basée sur des étapes connectées par des files d'événements.

SFZ Format ouvert décrivant des instruments échantillonnés.

SPF Sender Policy Framework : mécanisme indiquant les serveurs autorisés à envoyer des e-mails pour un domaine.

SSE Server-Sent Events : mécanisme permettant un flux de données serveur vers client via HTTP.

SSR Server-Side Rendering : technique de rendu d'interface effectuée côté serveur.

Stem Piste audio isolée correspondant à une source ou un instrument spécifique.

TLS Transport Layer Security : protocole assurant la sécurité des communications réseau par chiffrement.

VPO Virtual Playing Orchestra : bibliothèque d'instruments orchestraux au format SFZ.

VST Virtual Studio Technology : standard de plug-ins audio pour logiciels de production musicale.

WAV Waveform Audio File Format : format audio non compressé.

Webhook Mécanisme HTTP permettant de déclencher automatiquement une notification lors d'un événement.

Introduction générale

Cadre général

Ce travail de recherche a été mené au sein de la Faculté des Sciences de l'Université de Monastir, dans le cadre d'un projet de fin d'études en informatique. Il s'inscrit à l'intersection de l'ingénierie logicielle, de l'apprentissage automatique et des technologies audio, en réponse aux enjeux de production musicale assistée par ordinateur.

L'ère musicale de l'intelligence artificielle

L'essor de l'intelligence artificielle redessine les contours de l'industrie musicale mondiale, à une vitesse que peu de révolutions technologiques ont égalée. Ce qui relevait, il y a une décennie, de la recherche fondamentale s'est imposé comme une réalité économique mesurable : le marché mondial de l'intelligence artificielle appliquée à la musique a atteint 4,48 milliards de dollars en 2025 et devrait progresser à 5,55 milliards de dollars en 2026, pour culminer à 12,86 milliards d'ici 2030, avec un taux de croissance annuel composé de 23,4% [1]. Cette trajectoire reflète une transformation structurelle de la chaîne de valeur musicale, de la composition et de la production jusqu'à la distribution et la recommandation personnalisée. Portés par l'apprentissage profond, les architectures transformeurs et le traitement automatique du langage naturel, les systèmes génératifs ont reconfiguré les pratiques créatives, ouvrant des possibilités inédites pour le compositeur amateur comme pour le musicien professionnel.

Problématique

Les plateformes commerciales dominantes apprennent directement à partir de corpus audio bruts et synthétisent du son en l'absence de toute représentation symbolique (partition musicale (voir un exemple en annexe D), encodage harmonique ou représentation MIDI interprétable). Ces systèmes opèrent comme de véritables boîtes noires : leurs sorties sont audibles, mais leurs raisonnements internes demeurent impénétrables, rendant impossible toute justification du choix d'une note, toute reproductibilité d'un résultat et tout contrôle harmonique réel.

Cette opacité prive le compositeur de sa maîtrise : il ne peut ni corriger une décision musicale, ni réutiliser une génération comme point de départ, ni traduire le résultat en partition exploitable par de vrais musiciens.

La question centrale de ce travail est donc : **est-il possible de concevoir un système de génération musicale orchestrale restituant au compositeur le contrôle de sa création, tout en alliant légèreté computationnelle et transparence algorithmique ?** Le projet “Open Generative Studios” tente d’y répondre en articulant des services spécialisés, des modèles légers affranchis du GPU et une chaîne de traitement où chaque étape reste visible, modifiable et rejouable à volonté. La plateforme accompagne le compositeur dans une démarche itérative : chaque génération s’inscrit dans une conversation persistante, affinée au fil des échanges, et débouche sur une partition imprimable, passerelle concrète entre la création numérique et le monde musical réel, directement exploitable par des musiciens, des groupes ou des orchestres.

Valeur ajoutée du projet

Le projet se distingue par trois propriétés centrées sur l’expérience du compositeur : une composition itérative au sein d’une conversation persistante, permettant d’affiner la création au fil des échanges sans repartir de zéro, une partition imprimable exploitable directement par de vrais musiciens, ouvrant la création numérique vers le monde musical réel, et une architecture transparente où chaque étape du pipeline reste inspectable, modifiable et rejouable à volonté.

Philosophie et objectifs du projet

La plateforme repose sur une conviction : on ne devrait jamais faire confiance aveuglément à une machine pour créer de la musique. Plutôt que de déléguer le processus à un modèle opaque, le système rend visible chaque geste de la création : un prompt textuel devient une intention musicale structurée (voir l’annexe E), qui se décompose en couches MIDI de notes[M1] traversant un post-traitement maîtrisé, donnant naissance à un rendu audio orchestral et à une partition imprimable. Rien n’est escamoté, tout est traçable.

Chaque étape est un maillon que l’on peut saisir, modifier ou rejouer sans ébranler l’ensemble. C’est cette granularité qui change tout : elle restitue au compositeur sa capacité d’intervention à chaque moment du processus créatif.

De cette philosophie découlent cinq objectifs : la transparence de chaque étape de traitement, un contrôle musical fin sur la structure et les dynamiques[M2], une architecture modulaire aux responsabilités délimitées, la capacité à reproduire fidèlement une génération et à en retracer chaque étape, et un déploiement en production adossé à des mécanismes de sécurité, de facturation et de supervision.

Aperçu méthodologique : développement itératif centré architecture (Processus Unifié)

Le Processus Unifié (Unified Process – UP) est une méthodologie de développement logiciel itérative et incrémentale, centrée sur l'architecture, pilotée par les cas d'utilisation et dirigée par les risques [2]. Il vise à construire progressivement un système logiciel exécutable, tout en validant de manière précoce et continue les choix techniques et architecturaux majeurs. Cette approche permet de réduire significativement l'incertitude et de maîtriser les risques du projet dès les premières itérations, favorisant ainsi une plus grande prévisibilité et une meilleure maîtrise des coûts et des délais.

Le projet suit le Processus Unifié (UP), articulé en quatre phases de la conception à la mise en production :

- **Chapitre 1 : Création,** phase courte d'initialisation dédiée à la vision, au contexte et au socle technologique du système,
- **Chapitre 2 : Élaboration,** phase principale consacrée à l'architecture et au pipeline de génération musicale,
- **Chapitre 3 : Construction,** phase principale axée sur les services applicatifs, l'expérience utilisateur et la qualité,
- **Chapitre 4 : Transition,** phase courte de clôture couvrant le déploiement, l'exploitation et les perspectives du système.

Ce choix se justifie par la nature architecturale du système : un enchaînement de services dont la cohérence repose sur des contrats techniques fragiles : formats d'échange entre services, répartition des données, chargement de modèles d'apprentissage automatique et rendu audio instrumenté. La défaillance d'un seul de ces contrats suffit à invalider l'ensemble des traitements en aval.

La phase d'élaboration s'impose comme mécanisme de maîtrise du risque : elle exige une validation architecturale avant tout engagement de ressources, là où une méthode agile comme Scrum dilue ce risque sans garantir de clôture architecturale précoce, et où une méthode séquentielle comme le cycle en cascade le reporte en fin de projet.

Il en résulte un compromis pertinent, conjuguant itération contrôlée et jalons explicites, permettant de valider la viabilité du pipeline complet à petite échelle avant tout déploiement réel. Cette logique s'aligne sur les zones de risque du système et convient à un développement individuel orienté MVP : quatre phases pour dérisquer l'architecture et valider les hypothèses critiques, sans la surcharge des cadres conçus pour de grandes équipes. C'est dans ce cadre que s'ouvre le premier chapitre, consacré à la phase de Création : la vision, le contexte et le socle technologique qui conditionnent l'ensemble du développement ultérieur.

Chapitre 1

Création : Vision, contexte et socle technologique

1.1 Introduction

La phase de Création cadre le problème avant tout développement : les plateformes dominantes traitent l'audio brut, limitant le contrôle et la transparence. Une approche différente est adoptée, centrée sur le MIDI comme pivot, avec une architecture où chaque étape reste inspectable, modifiable et produit une partition exploitable. Ce chapitre présente le système, justifie les choix technologiques, décrit l'architecture SEDA [3] à six microservices et propose un bilan.

1.2 Analyse de l'existant

Le paysage de la génération musicale est dominé par des solutions audio-first centrées sur le prompt et le rendu direct. L'analyse suivante en synthétise les limites du point de vue du compositeur.

— **Suno** : génération prompt-to-audio grand public. La v4.5 met surtout en avant l'élargissement des styles, l'amélioration des voix et des morceaux pouvant atteindre huit minutes [4]. Des méta-tags de structure et d'instrumentation permettent d'orienter le modèle, mais n'agissent que comme de simples « signal weights » probabilistes (des suggestions d'orientation), sans garantie de reproductibilité ni d'artefact symbolique en sortie [5]. Le compositeur ne peut ni corriger une décision musicale précise, ni exporter une partition exploitable.

— **Udio** : variabilité résiduelle importante malgré l'ajout de la continuation et du remix par sections[M3]. Chaque génération repart de zéro sans mémoire de l'intention initiale, rendant toute démarche itérative laborieuse [6].

— **Google DeepMind (MusicLM/AudioLM)** : résultats de recherche à accès restreint, sans interface de compositeur ni outillage pour l'orchestration fine [7].

— **Meta (MusicGen)** : modèle open-source sans représentation symbolique native ni export de partition. Le compositeur obtient un rendu audio sans aucun levier pour en modifier la structure, l'instrumentation ou l'harmonie [8, 9, 10].

— **Stability AI (Stable Audio)** : contrôle audio avancé parmi les outils audio-first, mais les artefacts intermédiaires restent inaccessibles et aucune partition n'est produite [11].

— **AIVA** : exception partielle dotée d'un éditeur piano roll et d'un export MIDI natif sur tous les plans tarifaires. La plateforme reste néanmoins centrée sur des styles prédéfinis parmi plus de 250 genres, sans conversation persistante ni démarche itérative [12, 13].

— **Soundraw / Boomy** : solutions guidées par des templates, personnalisation limitée et sorties exclusivement audio, sans partition ni contrôle symbolique [14, 15].

Ces limites convergent vers trois lacunes structurelles du point de vue du compositeur, absence de contrôle itératif sur la création, impossibilité d'obtenir une partition exploitable par de vrais musiciens, et dépendance à un rendu audio opaque sans représentation symbolique :

— **Composition itérative** : Chaque génération s'inscrit dans une conversation persistante, affinée au fil des échanges, sans repartir de zéro à chaque prompt.

— **Partition exploitable** : Chaque composition débouche sur une partition imprimable, directement lisible par des musiciens, des groupes ou des orchestres.

— **Contrôle musical** : Chaque composition conserve une représentation symbolique de la structure, de l'instrumentation et des dynamiques, ajustable de manière déterministe, contrairement aux « signal weights » probabilistes des systèmes audio-first.

De plus, le système repose sur des modèles classiques de machine learning qui s'articulent autour de représentations symboliques MIDI, sans GPU à l'inférence, réduisant ainsi les coûts d'exploitation et la latence.

1.3 Vue d'ensemble du système

Le projet prend la forme d'une application web SaaS capable de transformer une intention musicale exprimée en langage naturel en une production audio accompagnée de sa partition. L'utilisateur soumet simplement une description depuis l'interface studio : la plateforme vérifie son identité et son solde de crédits, puis orchestre automatiquement une chaîne de services spécialisés qui se transmettent le travail les uns aux autres. Un modèle de langage interprète d'abord l'intention musicale, avant qu'une succession d'étapes dédiées ne construisent progressivement la composition, l'affinent, la synthétisent en audio et la mettent en partition. Tout au long du processus, l'interface studio reflète en temps réel l'avancement de la génération. Une fois terminés, l'audio et la partition sont directement accessibles depuis l'espace de conversation, tandis qu'un tableau de bord d'administration offre une vue d'ensemble sur les comptes, la consommation de crédits et la santé du système.

Le diagramme ci-dessous illustre l'indépendance entre l'application cliente et le tableau de bord administrateur. Il représente les cas d'usage d'un utilisateur avant (spectateur) et après authentification, leurs interactions avec le système, ainsi que les fonctionnalités réservées à l'administrateur.

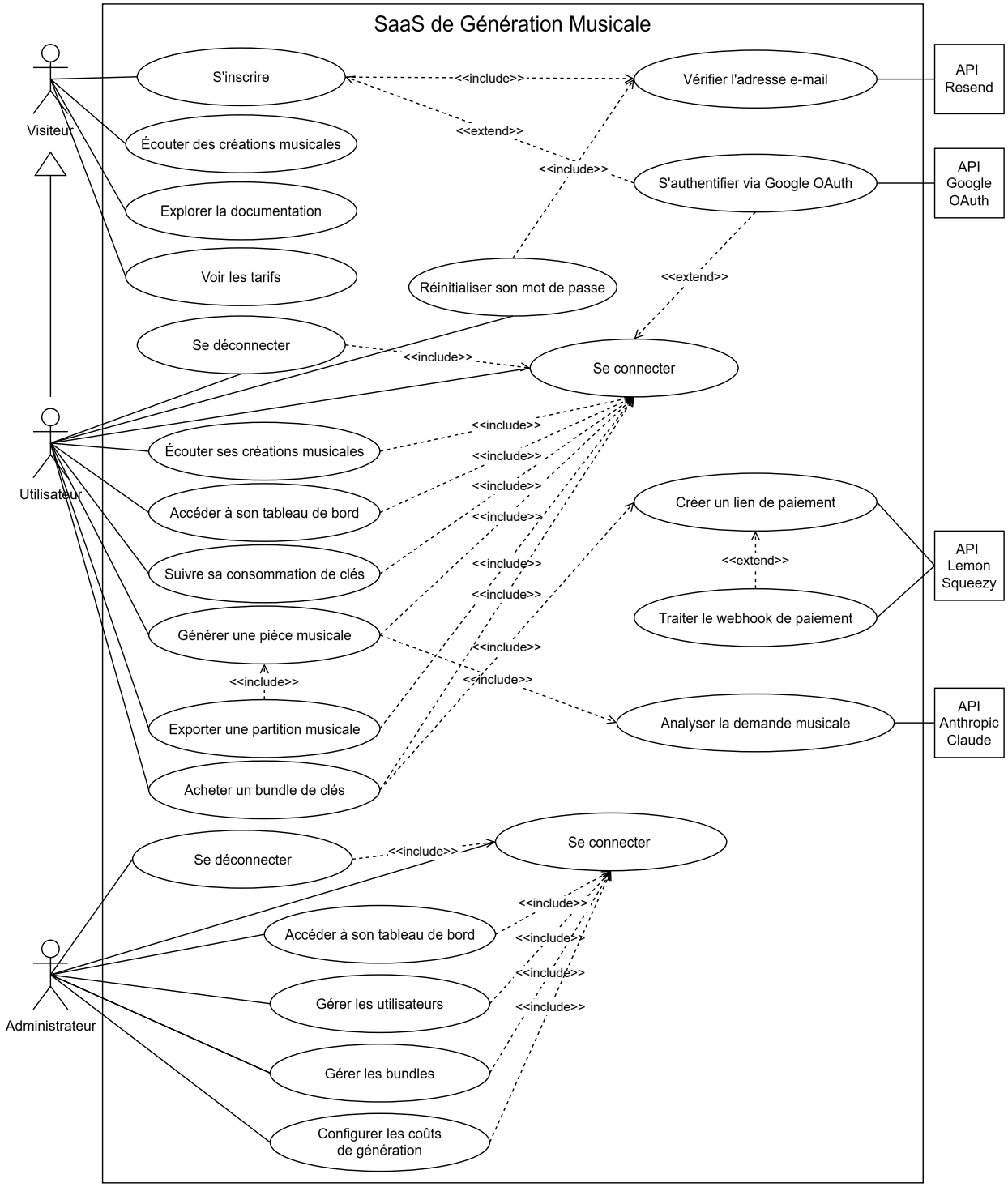


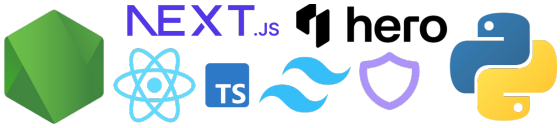
FIGURE 1.1 – Diagramme de cas d'utilisation : parcours utilisateur et services principaux

1.4 Justification des choix technologiques et architecturaux

Chaque outil répond à une contrainte précise du système. Cette section en détaille les choix.

1.4.1 Interface et applications web

Next.js App Router[T1] alimente deux applications aux besoins distincts. La première expose deux environnements : un rendu serveur (SSR) pour la découverte du projet, la documentation et l'authentification, optimisé pour le référencement, et un rendu client (CSR) pour l'expérience interactive en temps réel, là où s'orchestrent les interactions et où l'avancement du pipeline devient visible. Le tableau de bord administrateur repose sur le même rendu serveur, ce qui permet de garder les routes protégées hors de portée côté client. Les deux applications partagent la même base technique : TypeScript[T2], React[T3], Tailwind CSS[T4] et HeroUI[T5], ce qui évite toute divergence structurelle. Côté back-end, Node.js[T6] couvre les routes web, les sessions, les files Redis[T7] [16] et la facturation, alors que les traitements musicaux reviennent à des services Python[T8] dédiés.



1.4.2 Persistance et communication inter-services

MongoDB[T9] assure la persistance des données applicatives grâce à son modèle documentaire, naturellement aligné sur les structures imbriquées et les métadonnées complexes produites par le moteur de composition. Supabase[T10], basé sur PostgreSQL, gère l'identité, les crédits et les achats avec des garanties transactionnelles strictes. Redis sert de bus de files d'attente entre les étapes du pipeline et assure la limitation de débit des routes. Les artefacts finaux (WAV, PDF, MIDI) sont publiés sur DigitalOcean Spaces[T11], un stockage objet compatible S3.



1.4.3 Intelligence artificielle et machine learning

Claude Haiku[T12], retenu pour son JSON natif et son cache de prompts, convertit le prompt utilisateur en intention musicale structurée (JSON), faisant le lien entre le langage naturel et la génération [17]. Trois modèles, s'appuyant sur Scikit-learn[T13] et NumPy[T14] pour le traitement des représentations MIDI symboliques, prennent ensuite le relais [18]. Les artefacts sont produits sur Kaggle[T15] et versionnés sur Hugging Face[T16].



1.4.4 Traitement musical et audio

La bibliothèque mido[T17] gère la lecture, l'écriture et la transformation des fichiers MIDI [19]. Le rendu orchestral repose sur Sfizz[T18] et le format SFZ[T18], couvrant les familles classiques : cordes, vents, cuivres et percussions[M4], sans dépendre d'un DAW ni de plugins VST propriétaires [20, 21]. Pedalboard[T19] applique les effets audio depuis Python, notamment la réverbération, la compression et l'égalisation [22]. La conversion MIDI vers partition PDF est assurée par le CLI MuseScore[T20].



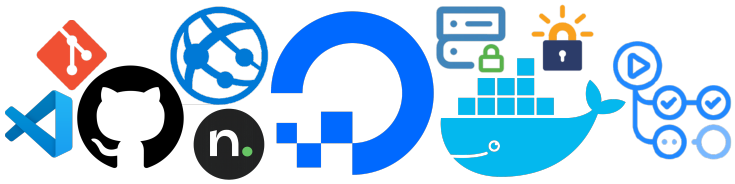
1.4.5 Sécurité

Les sessions reposent sur des jetons JWT[T21] en cookies HttpOnly, avec des mots de passe hachés via bcrypt et des gardes middleware. L'authentification via Google OAuth[T22] complète le flux standard. La limitation de débit s'appuie sur Redis, avec un repli en mémoire si indisponible.



1.4.6 Infrastructure et déploiement

Docker[T23] et Docker Compose[T23] fournissent un environnement reproductible en développement comme en production. GitHub Actions[T24] automatise le linting, les tests et le déploiement. En production, l'application cliente, les services Python et Redis s'exécutent sur un Droplet DigitalOcean, avec Caddy[T25] assurant la terminaison TLS reposant sur Let's Encrypt[T26]. Le domaine est géré via Name.com[T27], facilitant la configuration DNS et la délégation des sous-domaines. Le tableau de bord administrateur est déployé séparément sur Azure App Service[T28].



1.4.7 Facturation et communication

Lemon Squeezy[T29] gère les sessions de paiement et les webhooks créditant les comptes utilisateur, simplifiant la conformité fiscale en tant que Merchant of Record. Resend[T30] prend en charge les emails transactionnels de vérification et de réinitialisation.



1.4.8 Outillage de documentation

LaTeX[T31] et Grammarly[T32] assurent la rédaction du rapport. Les diagrammes UML sont produits avec Draw.io[T33] et les représentations de flux avec Mermaid[T34].



1.5 Architecture cible : pipeline à 6 microservices

L'architecture microservices structure le système en services indépendants, chacun responsable d'une étape du pipeline. Ce choix architectural constitue la décision fondatrice de la Création et conditionne l'ensemble du développement ultérieur.

1.5.1 Pourquoi une architecture microservices

Plutôt qu'un monolithe opaque, le système se déploie comme une chaîne de spécialistes où chaque service maîtrise son domaine, ignore le reste et passe le relais. Cette décomposition répond à quatre exigences concrètes :

— **Hétérogénéité assumée** : Chaque service adopte la pile la plus adaptée à sa responsabilité.
— **Déploiement chirurgical** : Un changement dans le moteur de composition ne nécessite pas de redéployer le service de rendu audio, ce qui réduit le risque de régression et accélère les cycles de livraison.
— **Scalabilité ciblée** : Les étapes les plus coûteuses en ressources peuvent être mises à l'échelle indépendamment des étapes légères.
— **Résilience cloisonnée** : La défaillance d'un service n'entraîne pas l'arrêt du système entier. Les files d'attente absorbent les retards et permettent la reprise après incident.

1.5.2 Pourquoi SEDA

Le traitement suit une séquence immuable : texte → intention musicale → notation symbolique brute → notation symbolique stabilisée → signal audio stéréophonique → partition imprimable, séquence que l'architecture érige en contrats explicites entre agents de traitement indépendants, reliés par des files d'attente. Ce patron convient naturellement à la génération musicale pour plusieurs raisons :

— **Observabilité** : Chaque file d'attente constitue un point de mesure et la progression du pipeline est traçable en temps réel depuis la passerelle de notification.
— **Découplage temporel** : Une étape lente ne bloque pas les précédentes et une étape en erreur peut être relancée sans reprendre le pipeline depuis le début.
— **Débogage graduel** : Les artefacts intermédiaires restent inspectables entre chaque étape, ce qui facilite le diagnostic des problèmes de qualité musicale.

1.5.3 Structure logique du système

Le diagramme de classes ci-dessous présente la structure logique retenue pour organiser les entités du domaine, les services de traitement et les relations principales entre composants.

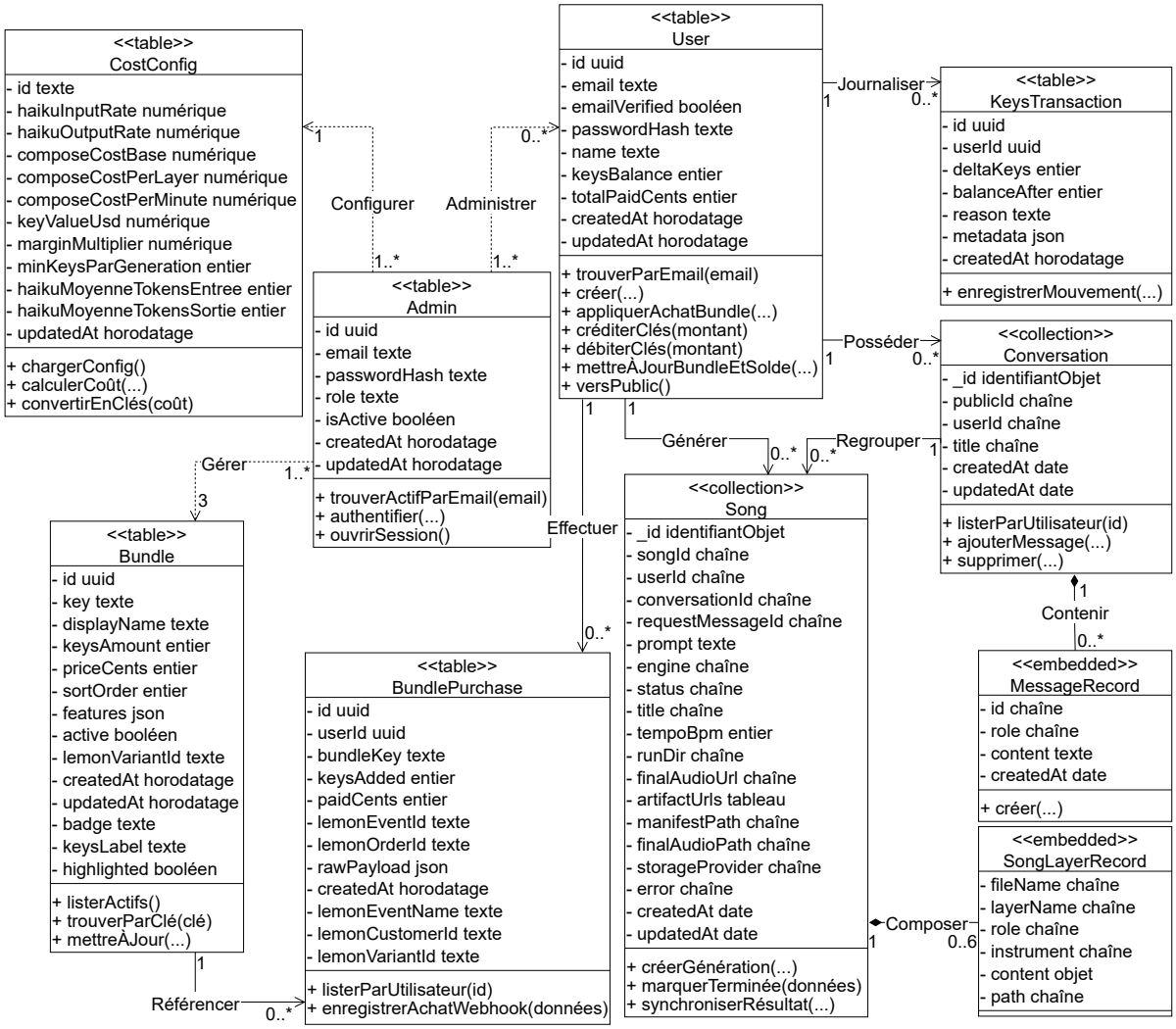


FIGURE 1.2 – Diagramme de classes : structure logique du système et relations clés

Les services opèrent sur ces entités sans partager d'état applicatif direct : les jobs transitent par les files Redis, les statuts par `notificationQueue` et les artefacts par le système de fichiers partagé.

1.5.4 Les six services du pipeline

Chaque service de traitement respecte une responsabilité unique, communique exclusivement via Redis et produit un artefact inspectable, tandis que la passerelle de notification assure l'observabilité transversale du pipeline.

1. **L'application web cliente** (Node.js/Next.js) constitue le point d'entrée unique de la plateforme et la seule interface exposée à l'utilisateur. Elle prend en charge l'authentification, la gestion des sessions et la vérification du solde de crédits avant d'accepter toute demande de génération.

Une fois la requête validée, elle construit le contexte du job et le soumet à la file `composeQueue`. En fin de pipeline, elle récupère les artefacts produits et les rend accessibles.

2. **Le moteur de composition** (Python) est le noyau cognitif du pipeline et le seul service autorisé à invoquer un modèle de langage. À réception d'un job, il soumet le prompt à Claude Haiku pour en extraire une intention musicale, puis valide les instruments demandés contre le catalogue VPO (Voir l'annexe B) afin de garantir leur compatibilité avec le rendu SFZ. Il pilote ensuite les trois modèles spécialisés : rythme, mélodie et expression, pour produire les couches MIDI brutes avant de publier vers la file `midiQueue`.
3. **Le service MIDI** (Python) est l'étape de stabilisation déterministe du pipeline. Il ne prend aucune décision créative mais garantit que les données symboliques issues de la composition sont propres et cohérentes avant d'atteindre le rendu audio. Il normalise les vélocités[M5] par section en s'appuyant sur l'intention, aligne et écrête la timeline, ferme les notes résiduelles non terminées et applique des fallbacks déterministes sur les couches vides. Il calcule un manifeste de traçabilité pour garantir l'intégrité des artefacts vers les services aval, puis publie vers la file `renderQueue`.
4. **Le service de rendu audio** (Python) est le seul service du pipeline à produire de l'audio. Il charge les affectations d'instruments, puis synthétise chaque piste MIDI individuellement via le moteur SFZ/Sfizz, qui s'appuie sur des échantillons orchestraux couvrant les familles cordes, vents, cuivres et percussions. Les stems obtenus sont ensuite assemblés en un mixage stéréo auquel sont appliqués les effets de mastering, réverbération, compression et égalisation, avant publication vers la file `notationQueue`.
5. **Le service de notation** (Python) matérialise la composition sous la forme d'une partition lisible, imprimable et partageable. Il consomme les fichiers MIDI post-traités et invoque le CLI MuseScore pour les convertir en partition PDF, en produisant un conducteur ainsi qu'une partition par instrument. Cette étape est déclenchée après le rendu audio via la file `notationQueue`, l'utilisateur devant disposer de son rendu audio avant de recevoir sa partition. Une fois la partition générée, le service publie les événements `notation :completed` puis `pipeline :completed` dans `notificationQueue`.
6. **La passerelle de notification** (Python) est le concentrateur transversal d'événements du système, orthogonal au pipeline de traitement. Elle ingère en continu les statuts publiés par les services de traitement, les horodate et les conserve dans un tampon (buffer) mémoire structuré. Elle expose au client un flux SSE en temps réel reflétant la progression du job étape par étape, ainsi que des routes de supervision permettant de consulter l'état des files, d'inspecter le contenu des files d'attente de lettres mortes (DLQ) et de vérifier la disponibilité de chaque service.

Les services de traitement publient des événements normalisés vers la passerelle, formant un contrat d'observabilité homogène sur l'ensemble du pipeline et permettant une détection proactive des goulots d'étranglement.

Les jobs en erreur sont isolés et acheminés vers une DLQ dédiée, offrant un espace de diagnostic structuré et la possibilité d'un rejeu ciblé sans perturber le flux principal. Cette séparation garantit qu'une anomalie isolée, qu'il s'agisse d'un timeout ou d'une indisponibilité transitoire, ne sature pas la file principale (backpressure) et ne dégrade pas la latence des jobs sains.

1.6 Mise en place de l'infrastructure initiale

1.6.1 Environnement Docker Compose

Un environnement local exécutable a été mis en place dès la phase de Création à l'aide de Docker Compose, regroupant l'application web cliente, les services Python du pipeline, Redis et le réseau inter-services au sein d'une configuration unifiée. L'intégralité de ces composants démarre en une seule commande, garantissant la reproductibilité de l'environnement de développement et éliminant tout problème de dépendances système.

1.6.2 Validation initiale de la communication par files

Le flux minimal d'ingestion et de transfert par file d'attente a été validé avec la sémantique Redis BRPOP/LPUSH [16]. Cette validation a confirmé que le mécanisme de communication inter-services fonctionne correctement : un message poussé par un service est consommé de manière fiable par le suivant. Le comportement bloquant de BRPOP garantit qu'un service en attente ne consomme pas de ressources inutilement et se réveille dès qu'un message devient disponible. La persistance des messages dans les files assure qu'aucune donnée n'est perdue lors du redémarrage d'un service : le traitement reprend automatiquement là où il s'était arrêté, sans intervention manuelle ni perte d'artefact intermédiaire. Ce mécanisme constitue le socle de résilience sur lequel repose l'ensemble du pipeline.

1.7 Bilan de la Création

1.7.1 Résultats atteints

La phase de Création a validé les fondations du projet : le pipeline est exécutable, les frontières de services sont définies, la communication par files d'attente Redis fonctionne de bout en bout et l'infrastructure Docker Compose permet un démarrage reproductible de l'ensemble du système.

1.7.2 Limitations identifiées

La qualité musicale finale n'est pas encore validée à ce stade. La Création a privilégié le cadrage architectural, les contrats techniques et la faisabilité d'orchestration plutôt que l'optimisation des sorties audio, conformément à l'objectif de la phase.

1.7.3 Défis reportés vers l'Élaboration

La faisabilité est validée. L'Élaboration doit transformer ces frontières techniques en services fiables, observables et capables de produire des artefacts musicaux exploitables.

Ce passage de l'esquisse à l'implémentation exige que chaque hypothèse validée devienne une contrainte concrète, tenue par du code et vérifiable à l'exécution :

- **Implémentation des six services** : Chaque composant doit passer du rôle architectural prévu à une responsabilité logicielle concrète, avec ses entrées, ses sorties et ses erreurs propres.
- **Stabilisation des contrats de messages** : Les charges utiles Redis, les événements de progression et les conventions d'échec doivent garantir l'enchaînement fiable des services.
- **Validation du flux de bout en bout** : Le système doit transformer un prompt utilisateur en fichiers MIDI, en rendu audio et en partition PDF sans intervention manuelle.
- **Intégration de la pile de génération** : Les modèles de rythme, de mélodie et d'expression doivent être reliés au moteur de composition pour produire des couches musicales cohérentes avec l'intention initiale.
- **Observabilité minimale** : Les statuts, les erreurs et les artefacts intermédiaires doivent rendre le pipeline inspectable et déboguable à chaque étape.

Ces défis marquent le passage d'une preuve de faisabilité architecturale à une chaîne de production musicale distribuée, mesurable et validable.

1.8 Conclusion

La Création pose les fondations techniques du projet et démontre la faisabilité de l'architecture. La phase d'Élaboration détaille l'implémentation, service par service, et valide le pipeline à travers un parcours utilisateur minimal.

Chapitre 2

Élaboration : Architecture et pipeline de génération musicale

2.1 Introduction

La phase de Création a posé les fondations et identifié les défis structurants de l'architecture. La phase d'Élaboration les transforme en contraintes concrètes, implémentées et vérifiables à l'exécution. C'est le moment où les hypothèses architecturales sont testées et où les risques majeurs sont levés avant tout investissement en qualité et en expérience utilisateur.

2.2 Architecture du système

2.2.1 Mise en œuvre SEDA

Les six services définis en Création ont été déployés et interconnectés conformément à la topologie établie, sans modification structurelle majeure. L'implémentation a toutefois révélé deux ajustements concrets : la nécessité d'un timeout explicite sur BRPOP pour éviter les blocages silencieux lors des redémarrages de services, et l'enrichissement des messages pour permettre la distinction entre traitement initial et rejeu depuis la DLQ. Ces ajustements ont été intégrés dans les contrats de messages sans altérer la topologie prévue.

2.2.2 Contrats de files d'attente et résilience

L'Élaboration a figé les charges utiles circulant entre services. Chaque job embarque un identifiant unique, les métadonnées de l'intention musicale et les chemins vers les artefacts de l'étape précédente. Ce contrat de message uniforme garantit qu'un service peut être remplacé ou redémarré sans altérer la lisibilité des données pour le service suivant.

Les jobs placés en DLQ conservent leur contexte d'échec complet : message d'erreur, horodatage et numéro de tentative. Cette traçabilité permet un diagnostic précis et un rejeu ciblé sans perturber le flux principal.

La figure ci-dessous représente la topologie déployée : chaque service apparaît comme un nœud autonome, relié aux autres exclusivement par des files Redis.

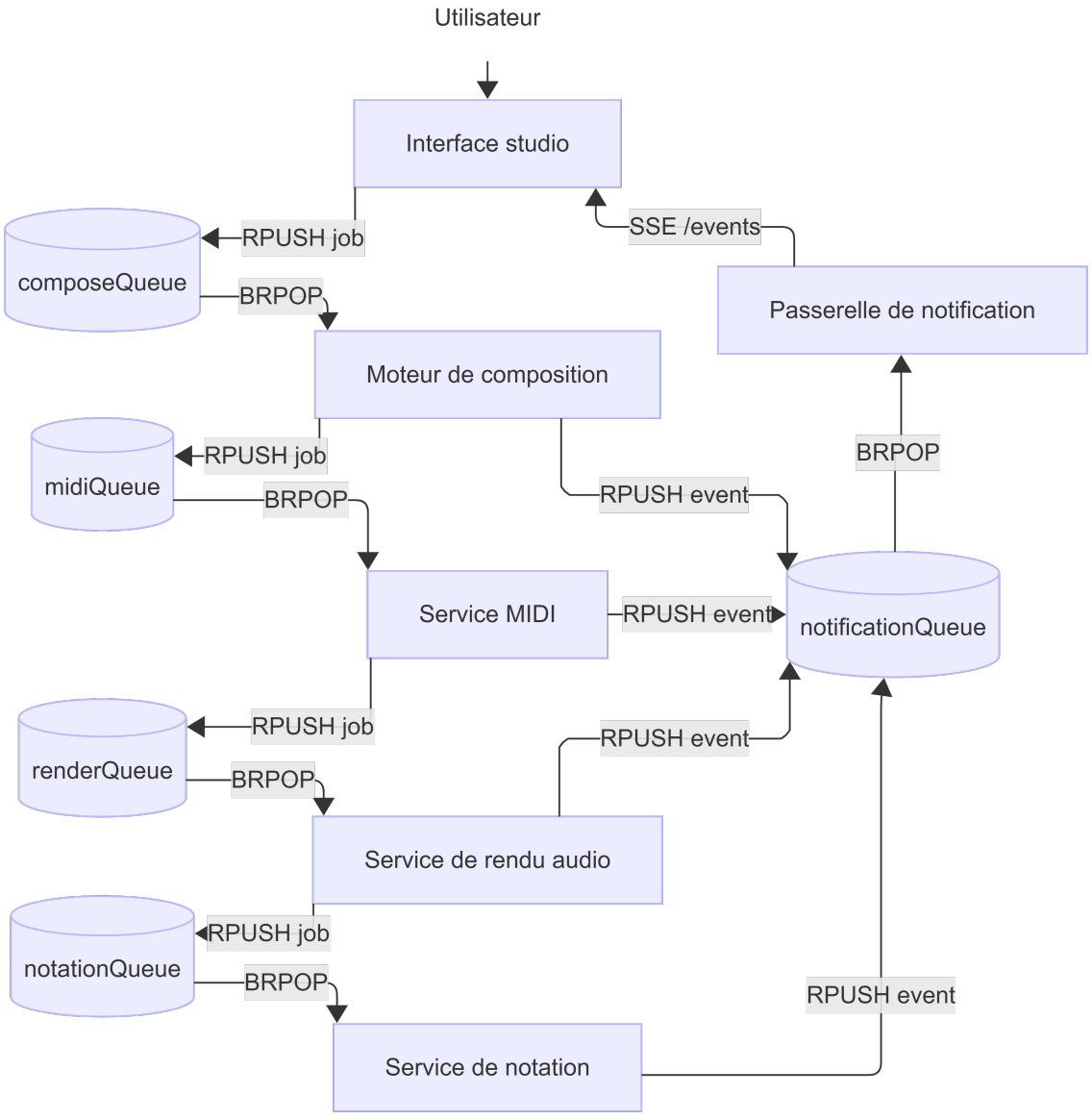


FIGURE 2.1 – Représentation de l'architecture SEDA et interactions entre les services

Le découplage par files d'attente assure une forte indépendance opérationnelle.

2.2.3 Flux bout en bout

Le pipeline transforme un prompt texte libre en un ensemble d'artefacts musicaux structurés à travers cinq étapes successives, chacune produisant un artefact inspectable indépendamment.

Le processus devient ainsi entièrement traçable et déboguable étape par étape :

— Prompt texte → intention sous forme d'objet JSON.

— Intention → génération de MIDI bruts via la pile de modèles de composition.

— MIDI bruts → MIDI post-traités.

— MIDI post-traités → audio mixé : pistes séparées (stems), puis mix final WAV.

— MIDI → notation PDF.

2.3 Service de passerelle applicative

L'application web cliente constitue la passerelle utilisateur : elle applique l'authentification, les contrôles de propriété, les politiques de bundle et le coût de génération. Seul point d'accès aux données, toutes ses routes sont protégées par vérification de session Supabase, et l'identité du propriétaire est contrôlée avant toute opération, garantissant une isolation des données entre utilisateurs.

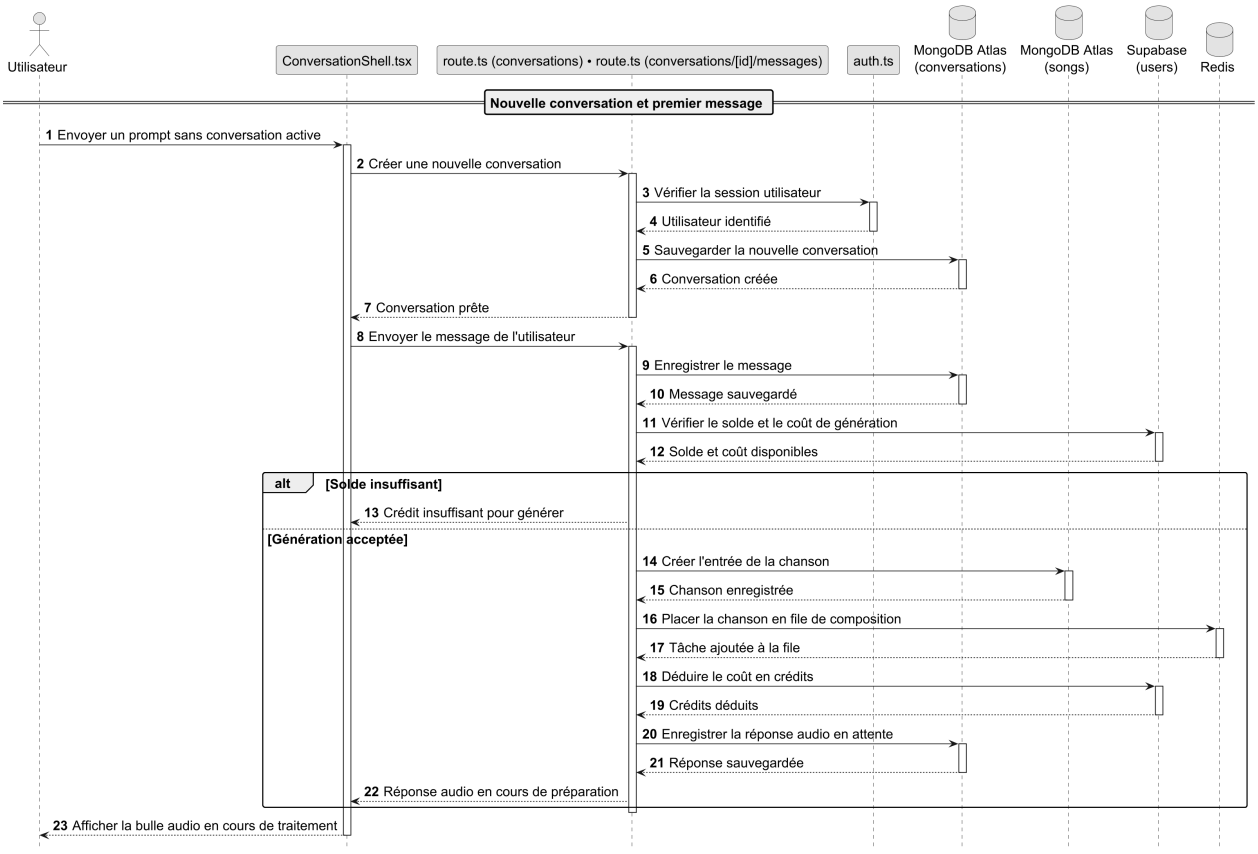


FIGURE 2.2 – Diagramme de séquence : génération, envoi du premier message et mise en file de composition

L'envoi du premier message déclenche l'initialisation d'une génération : l'API valide la session et le contexte de la requête, calcule dynamiquement le coût à partir de la table de configuration, combinant un coût fixe d'extraction d'intention (Haiku) et un coût variable dépendant du nombre de couches ainsi que de la durée de la composition, puis lance le traitement asynchrone du job.

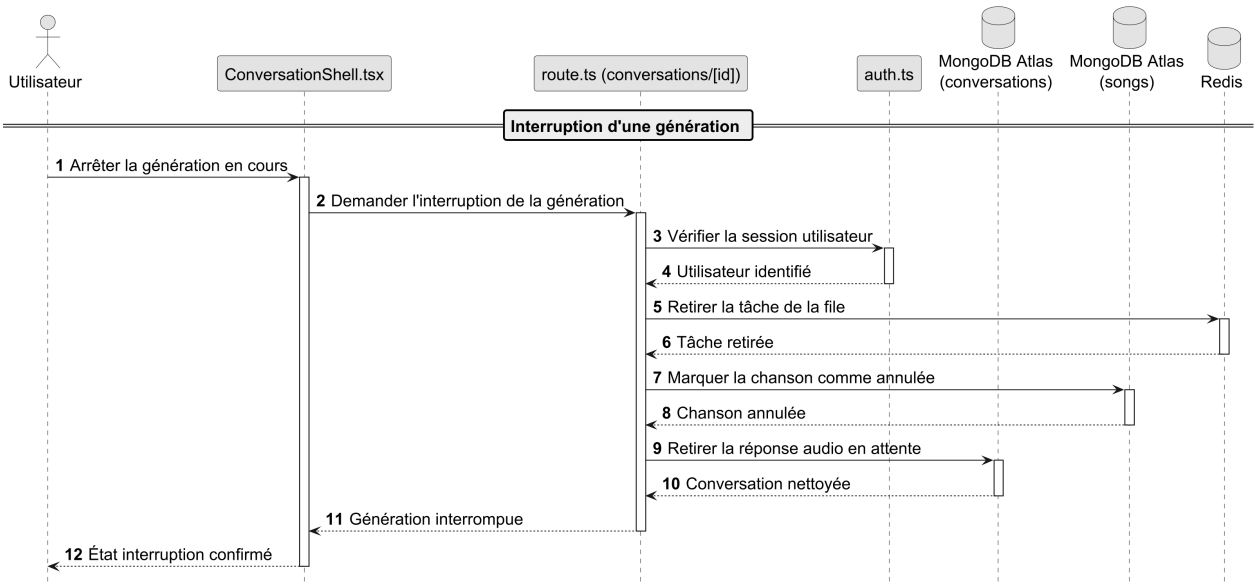


FIGURE 2.3 – Diagramme de séquence : interruption, annulation d'une génération en cours

L'interruption d'un job en cours suit une chaîne de sécurité et de nettoyage : vérification de session, retrait de la tâche Redis, annulation de la chanson et purge de la réponse en attente.

2.4 Service de composition

Le moteur de composition est l'étape la plus riche du pipeline. Il consomme `composeQueue` et enchaîne quatre phases : extraction d'intention, résolution instrumentale, génération musicale multi-modèles et persistance des artefacts.

2.4.1 Extraction d'intention et résolution instrumentale

À réception d'un job, le moteur soumet le prompt utilisateur à Claude Haiku, qui retourne une structure d'intention au format JSON contenant tempo[M6], tonalité[M7], genre, sections et liste d'instruments [17]. L'intention est ensuite validée contre le catalogue VPO afin de garantir que chaque instrument demandé correspond à une entrée réelle, dotée d'une tessiture[M8] et d'un mapping SFZ exploitables par les stages aval, avant d'engager la génération MIDI [21].

2.4.2 Pile de génération musicale : une approche hybride

La génération MIDI brute repose sur une architecture tripartite, chaque module assumant une dimension distincte de l'expression musicale. Ce paradigme hybride est délibéré : aucun modèle unique ne couvre simultanément la structure rythmique, la conduite mélodique et l'expressivité dynamique, et les contraintes matérielles (entraînement et inférence sur CPU sans GPU) ont orienté le choix vers des architectures légères et modulaires plutôt que vers un grand modèle auto-régressif.

Les corpus retenus présentent une complémentarité structurelle fondée sur des propriétés distinctes :

MAESTRO (Google Magenta) : corpus de référence en génération musicale symbolique, constitué dans le cadre du projet *Magenta* de Google et reconnu pour la rigueur de ses performances pianistiques MIDI alignées sur l'interprétation humaine [23].

Lakh MIDI Dataset : jeu de données MIDI largement exploité dans la littérature en génération musicale, distingué par l'étendue de sa couverture stylistique, la variété de ses motifs mélodiques et la richesse de ses structures rythmiques [24].

Des corpus spécialisés tels que Groove et ASAP ont en outre servi de références méthodologiques pour situer les exigences rythmiques et score-performance, sans toutefois répondre à l'objectif d'orchestration multi-instrumentale visé [25, 26].

Cette complémentarité conjugue expressivité et amplitude stylistique dans un même cadre d'entraînement. MAESTRO fournit des performances pianistiques de haute précision, intégrant les nuances de vélocité, de dynamique et les subtilités du phrasé[M9] interprétatif. Le Lakh MIDI Dataset apporte un répertoire étendu de genres pour modéliser les distributions mélodiques et les structures rythmiques récurrentes, permettant au modèle de généraliser au-delà d'un registre[M10] unique.

a. Composante expressive

Le Random Forest [27] a été retenu pour la composante expressive car il modélise efficacement des relations non linéaires entre contexte musical, vélocité et micro-variation de durée sans dépendre d'un grand modèle séquentiel. Son apprentissage via « bagging » réduit la variance et reste robuste sur des données tabulaires issues de caractéristiques musicales, ce qui correspond au format extrait de MAESTRO. Dans cette configuration, 200 arbres entraînent deux têtes indépendantes : la première prédit la vélocité MIDI (0–127), la seconde prédit le delta de durée δ, défini par :

$$\delta = \frac{\text{durée réelle} - \text{durée quantifiée}}{\text{durée quantifiée}}, \qquad \delta \in [-0{,}9;\ 2{,}5],$$

afin de supprimer les artefacts de timing extrêmes.

— **Données et séparation train/test :**

TABLEAU 2.1 – Données et séparation train/test pour la composante expressive

Ensemble	Fichiers MIDI	Notes supervisées
Entraînement	800 (80 %)	600 000
Test	200 (20 %)	150 000
Total	1 000	750 000

Le script parcourt 1 000 fichiers MIDI MAESTRO (0 échec de lecture, 5 494 277 notes au total). La séparation est effectuée au niveau des fichiers (toutes les notes d'un même morceau restent dans le même ensemble), évitant toute fuite entre notes d'un même enregistrement.

Les normaliseurs sont ajustés uniquement sur l'entraînement puis appliqués au test, préservant l'indépendance des métriques.

— **Vecteur de caractéristiques (21 dimensions)**

Le tableau suivant décrit chaque variable d'entrée et son rôle :

TABLEAU 2.2 – Vecteur de caractéristiques de la composante expressive (21 dimensions)

Variable	**Signification**
Hauteur[M11]	Valeur MIDI de la note courante (0–127).
Intervalle[M12]	Saut signé de hauteur depuis la note précédente.
Intervalle absolu	Valeur absolue du saut (ampleur sans direction).
Hauteur précédente	Valeur MIDI de la note immédiatement avant.
Hauteur suivante	Valeur MIDI de la note immédiatement après.
Position dans la séquence	Rang normalisé de la note dans la piste ($\in [0, 1]$).
Densité locale	Nombre de notes dans une fenêtre de ± 6 notes.
Classe de hauteur[M13]	Chroma (0–11, indépendant de l'octave).
Octave[M14]	Registre octaviant de la note.
Registre aigu	Indicateur binaire : hauteur ≥ 72 (mi aigu et au-dessus).
Registre grave	Indicateur binaire : hauteur ≤ 48 (ut grave et en-dessous).
Durée précédente	Durée quantifiée de la note précédente.
Durée courante	Durée quantifiée de la note (exclue de la tête δ).
Ratio de durée	Rapport durée courante sur la moyenne mobile des durées.
Contexte de dynamique	Moyenne mobile exponentielle des intensités (α=0,15).
Écart d'attaque[M15]	Intervalle de temps entre deux attaques consécutives.
Section formelle	Identifiant de la section structurelle (0 = intro à 4 = outro).
Position dans la section	Rang normalisé de la note dans sa section ($\in [0, 1]$).
Contexte de durée	Moyenne mobile exponentielle des durées réelles (α=0,10).
Distance de fin de phrase	Proportion restante jusqu'à la fin de la phrase courante.
Durée suivante	Durée quantifiée de la note qui suit.

— **Hyperparamètres :**

- **Ensemble :** 200 arbres de décision entraînés en parallèle via « bagging », chacun sur un sous-échantillon aléatoire distinct des données et des variables. Ce nombre résulte d'un arbitrage entre stabilité prédictive et coût d'inférence.
- **Tête vélocité :** profondeur maximale 18, taille minimale de feuille 3. Ce réglage capte les variations de vélocité observées dans MAESTRO tout en maintenant la généralisation.

  - **Tête δ-durée :** profondeur maximale 14, taille minimale de feuille 6. La structure est volontairement moins profonde que pour la vélocité : la cible est plus bruitée et intrinsèquement plus variable.
  - **Évaluation :** score « out-of-bag » activé sur les deux têtes, fournissant une validation croisée implicite sans ensemble dédié. Chaque arbre est appris sur un sous-échantillon aléatoire des données d'entraînement, garantissant une estimation robuste et non biaisée des performances.

— **Métriques de validation**

TABLEAU 2.3 – Métriques de validation de la composante expressive

Métrique	**Interprétation**	**Vélocité**	**δ-durée**
R^2 (test)	Fraction de variance expliquée sur les données non vues	0,616	0,377
R^2 (train)	Même indicateur sur l'entraînement	0,760	0,431
OOB R^2	Validation croisée implicite sans ensemble dédié	0,600	0,375
MAE (test)	Erreur absolue moyenne sur l'échelle cible	8,70	0,194

Lecture des chiffres

La MAE de vélocité ($8{,}70$) porte sur une échelle de 127 niveaux : elle représente une erreur inférieure à 7 % de la plage totale. L'accord OOB / test ($0{,}600$ vs $0{,}616$) confirme l'absence de fuite de données. L'écart R^2 train / test de la tête δ ($0{,}431$ vs $0{,}377$) indique un bon équilibre biais/variance après optimisation des hyperparamètres. L'importance dominante du contexte de dynamique (**63 %**) confirme que l'intensité d'une note dépend fortement de la dynamique immédiatement précédente.

b. Composante mélodique

La chaîne de Markov d'ordre 3 a été retenue pour la mélodie car elle capture un contexte local suffisant pour orienter la phrase musicale sans imposer le coût d'un modèle auto-régressif profond. Elle est aussi naturellement probabiliste : la température permet de doser la prévisibilité et la variation, deux propriétés essentielles pour générer plusieurs lignes plausibles à partir d'une même intention. Le modèle prédit le prochain degré[M16] de gamme à partir des trois degrés précédents : $P(d_n \mid d_{n-3},\, d_{n-2},\, d_{n-1})$. Les transitions du corpus Lakh MIDI sont comptabilisées pour les ordres 1 à 3. À l'inférence, le moteur exploite le plus long contexte disponible puis se replie vers un ordre inférieur si nécessaire. Le degré échantillonné est projeté en hauteur MIDI selon la gamme et la tessiture de l'instrument, dans l'esprit de l'anticipation musicale de Huron [28].

Le lissage de Laplace ($\alpha = 0{,}25$) garantit des probabilités non nulles sur les contextes rares, assurant une couverture totale à l'inférence.

— **Données et séparation train/test :**

Le corpus **Lakh MIDI** est utilisé, avec jusqu'à 2 500 fichiers candidats par genre lorsque le corpus filtré le permet [24]. La séparation est effectuée au niveau des **fichiers** (80/20) par genre. Les hauteurs MIDI sont converties en degrés selon le mode tonal du genre.

TABLEAU 2.4 – Données et séparation train/test pour la composante mélodique

Ensemble	**Fichiers/genre (moy.)**	**Contextes trigrammes**
Entraînement	∼548	343 (= 7^3 pour gamme 7 notes)
Test	∼139	343

12 genres couverts : cinematic, orchestral, classical, jazz, blues, rock, pop, ambient, electronic, folk, r-and-b, hip-hop. Chacun est associé à un mode tonal spécifique qui définit l'ensemble des degrés vers lesquels la chaîne de Markov peut transiter.

— **Interprétation de la perplexité**

La perplexité mesure combien de notes le modèle hésite en moyenne lors de chaque prédiction :

$$\text{PP} = 2^{-\frac{1}{N}\sum_i \log_2 p(d_i|d_{i-3},d_{i-2},d_{i-1})}$$

Une gamme diatonique[M17] contient 7 notes distinctes (do, ré, mi, fa, sol, la, si). Elle peut donc être parcourue de "do" à "do", de "ré" à "ré", ou depuis n'importe quelle note de départ. Le modèle choisit toujours parmi ces 7 degrés, ce qui fixe la référence du pire cas à PP $= 7$.

TABLEAU 2.5 – Interprétation de la perplexité par intervalle

Perplexité	**Comportement musical**
1–2	Trop répétitif / robotique
2–3	Très structuré, peu de variété
3–4	Structuré mais limité
4–5	**Équilibre optimal : créativité et structure**
5–6	Varié mais peu cohérent
6–7	Aléatoire pur (ligne de base)

Le modèle réduit l'incertitude de 7 à $\approx 4{,}86$ en moyenne : il a appris la structure musicale tout en préservant la variété, privilégiant les enchaînements fréquents sans verrouiller la mélodie sur une suite prévisible.

— **Métriques par genre (ensemble de test)**

TABLEAU 2.6 – Métriques par genre pour la composante mélodique (ensemble de test)

Genre	Perplexité ↓	Précision top-1 ↑	Couverture
blues	4,29	38,9 %	100 %
cinematic	4,71	38,2 %	100 %
electronic	4,72	37,0 %	100 %
r-and-b	4,72	37,3 %	100 %
hip-hop	4,77	37,7 %	100 %
rock	4,91	36,2 %	100 %
jazz	4,92	37,1 %	100 %
classical	5,00	37,3 %	100 %
pop	5,02	36,6 %	100 %
ambient	5,05	35,9 %	100 %
folk	5,05	35,9 %	100 %
orchestral	5,12	35,2 %	100 %

Lecture des chiffres

La **précision top-1** ($\sim$37 %) doit être comparée à la référence aléatoire de $1/7 \approx 14\,\%$: le modèle est **2,6$\times$ plus précis que le hasard**.

c. Composante rythmique

Le K-Means a été retenu pour la composante rythmique car les mesures peuvent être décrites comme des vecteurs d'attaque binaires, un format bien adapté au regroupement non supervisé. Il permet d'extraire des familles de grooves[M18] sans annotation manuelle, ce qui convient à la diversité du corpus Lakh MIDI et limite le coût d'entraînement. L'algorithme regroupe ainsi les profils récurrents [29] : certaines mesures[M19] ont des attaques denses et régulières, tandis que d'autres laissent davantage d'espace, permettant au moteur de choisir un motif adapté à l'énergie demandée.

— **Données et séparation train/test :**

230 fichiers Lakh MIDI par genre, découpés en mesures (bar-level split 80/20) :

TABLEAU 2.7 – Données et séparation train/test pour la composante rythmique

Ensemble	Mesures/genre	Total (12 genres)
Entraînement	20 050	240 600
Test	5 012	60 144
Total	25 062	300 744

— **Métriques par genre (train et test)**

TABLEAU 2.8 – Métriques par genre pour la composante rythmique (train et test)

Genre	k	Silhouette (entr.) ↑	Silhouette (test) ↑	MSE (entr.) ↓	MSE (test) ↓
orchestral	70	0,749	0,719	0,268	0,290
rock	60	0,723	0,719	0,305	0,299
hip-hop	60	0,708	0,708	0,306	0,308
classical	60	0,707	0,704	0,303	0,310
cinematic	60	0,699	0,716	0,305	0,297
electronic	60	0,696	0,708	0,303	0,302
pop	50	0,691	0,689	0,339	0,357
r-and-b	50	0,695	0,668	0,335	0,365
folk	50	0,678	0,692	0,342	0,349
ambient	50	0,670	0,680	0,341	0,354
jazz	50	0,686	0,691	0,350	0,330
blues	50	0,663	0,660	0,346	0,355

Lecture des chiffres

La silhouette ($\in [-1, +1]$) mesure la séparabilité des clusters : une valeur $> 0{,}5$ indique des groupes bien distincts.

d. Retour d'expérimentation sur les entraînements

Les modèles ont été entraînés et comparés à plusieurs reprises avant de retenir cette configuration. L'augmentation massive du nombre de notes MAESTRO pour le Random Forest n'a pas amélioré la qualité mesurée : le R^2 de vélocité sur test restait autour de $0{,}61$ et le modèle produit devenait beaucoup plus volumineux, ce qui a conduit à conserver la version plus légère. À l'inverse, la composante mélodique a nettement bénéficié du passage d'une chaîne de Markov d'ordre 2 à une chaîne d'ordre 3 avec repli vers les ordres inférieurs : la perplexité moyenne est passée d'environ $5{,}29$ à $4{,}86$ et la précision top-1 d'environ $33\,\%$ à $37\,\%$. Les essais ont ainsi montré que l'ordre 2 capturait insuffisamment le contexte mélodique, tandis qu'un ordre plus élevé augmentait la rareté des contextes sans gain proportionnel, l'ordre 3 constituant donc le meilleur compromis observé entre mémoire musicale et généralisation.

2.4.3 Articulation des modèles dans le flux de composition

Ces composants ne fonctionnent pas comme trois sorties indépendantes, mais comme une chaîne de décision du moteur de composition.

Le service reçoit l'intention complète du job, résout les instruments à partir du catalogue VPO, puis prépare les paramètres nécessaires à la génération des couches MIDI.

La composante rythmique propose les attaques, la composante mélodique produit les degrés et la composante d'expression prédit les intensités et les durées. Le résultat de cette articulation est une matière musicale structurée, prête à être écrite en artefacts intermédiaires.

2.4.4 Persistance et transmission à l'étape suivante

Après génération, le service persiste les artefacts intermédiaires du job, publie les métadonnées nécessaires au suivi, puis transmet le contexte d'exécution au service suivant via la file de traitement MIDI. Les événements d'erreur, les traces d'exception et le contenu des DLQ sont persistés en MongoDB, offrant un journal structuré et horodaté pour la relecture et le rejeu ciblé des jobs en échec.

2.5 Service de post-traitement MIDI

Ce service ne compose pas. Il stabilise et normalise les sorties symboliques issues de la composition afin de garantir un matériau cohérent pour le rendu audio et la notation. Le traitement actuel applique :

— Ajustements de vélocité par section à partir de la structure d'intention
— Normalisation et écrêtage de la timeline et fermeture sûre des notes actives
— Vérification des couches produites et calcul d'empreintes de contrôle pour tracer les artefacts transmis

En sortie, le service fournit un jeu MIDI stable et reproductible, ce qui réduit les artefacts de rendu et simplifie l'analyse des incidents en aval. Le résultat est ensuite transmis vers `renderQueue`.

2.6 Service de rendu audio

Ce service transforme les données MIDI post-traitées en artefacts audio exploitables. Il consomme la file `renderQueue`, charge les affectations d'instruments depuis le manifeste de configuration, rend chaque stem de manière indépendante via le moteur SFZ/Sfizz, puis assemble le mix final stéréo en appliquant les pondérations de volume et de panoramique définies par l'utilisateur [20, 21]. Les effets de mastering (réverbération, compression, égalisation), le mix bus et les niveaux de sortie sont entièrement paramétrables via la configuration du job. Le backend SFZ/Sfizz est utilisé en priorité pour ses performances natives et sa fidélité timbrale, tandis qu'un fallback Python assure la continuité de service en cas d'indisponibilité [22]. Cette étape isole la complexité de synthèse du reste du pipeline et facilite le dimensionnement indépendant de la charge CPU liée au rendu.

2.7 Service de notation musicale

Ce service convertit les données symboliques issues du pipeline en partition musicale structurée, destinée à la consultation, à l'impression et à l'export documentaire. Il consomme `notationQueue`, invoque MuseScore CLI et publie la complétion du pipeline.

Pour l'utilisateur, cette étape matérialise une approche pédagogique inversée : plutôt que d'apprendre la notation pour composer, l'utilisateur génère d'abord une composition à partir d'une intention exprimée en langage naturel, puis en découvre la transcription formelle sous forme de partition.

Ce renversement du flux traditionnel abaisse la barrière d'entrée à la théorie musicale, et transforme chaque génération en support d'apprentissage concret, partageable et rejouable par tout musicien.

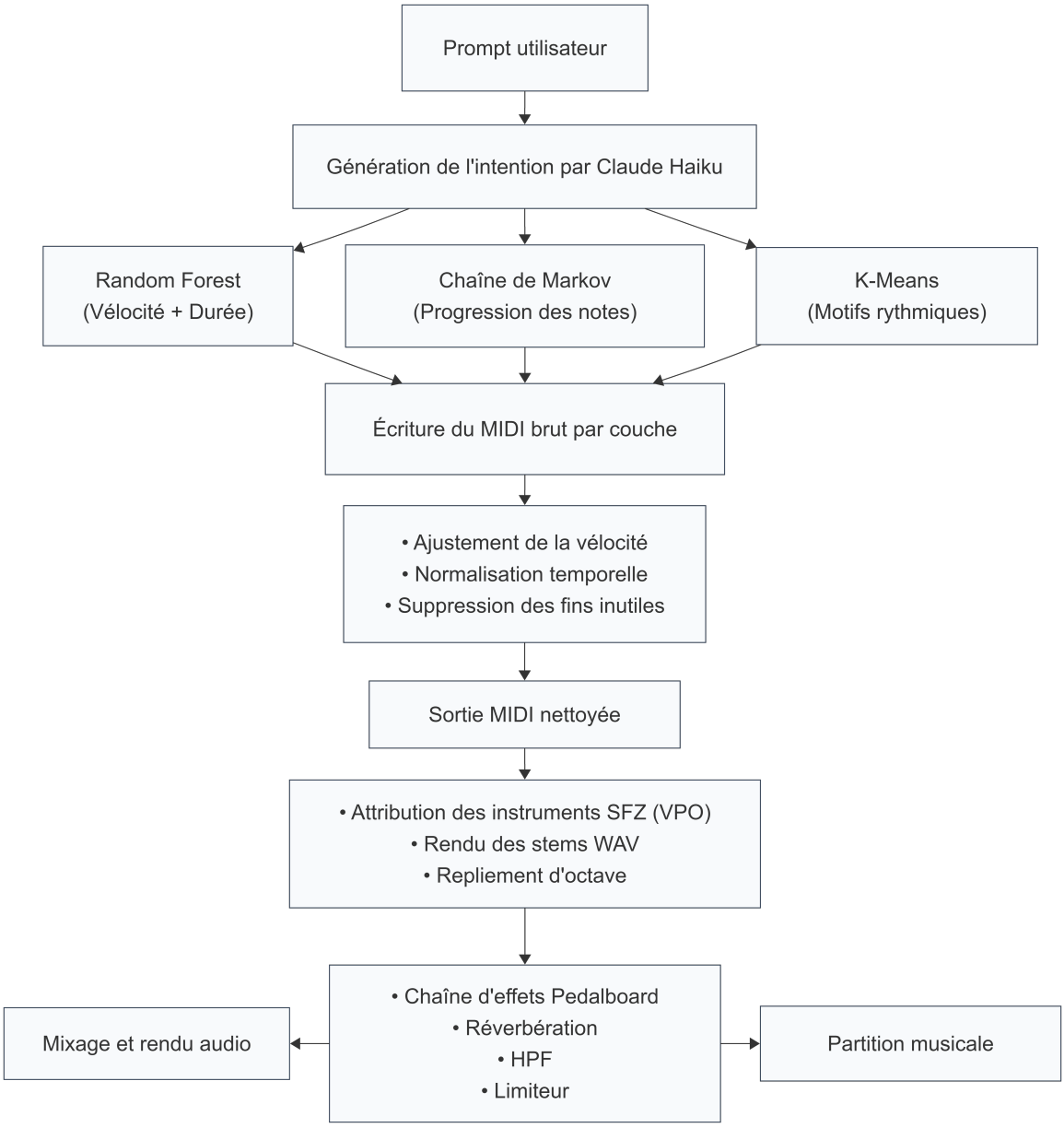


FIGURE 2.4 – Vue consolidée du pipeline de génération musicale

2.8 Service de notification

Ce service centralise la télémétrie opérationnelle du pipeline, ingère les événements depuis Redis et HTTP, et les expose en temps réel au client ainsi qu'aux outils de supervision [16, 30].

Il conserve un buffer mémoire, persiste les logs et expose les endpoints : /health, /notify, /events (SSE), /jobs (profondeur des files et DLQ). En pratique, il joue un rôle clé pour l'observabilité, la détection rapide des blocages et le diagnostic des échecs transverses entre services.

2.9 Bilan de l'Élaboration

2.9.1 Résultats atteints

Le pipeline est opérationnel de bout en bout et produit les artefacts principaux (MIDI, audio, notation) avec traçabilité par files d'attente et notifications. La pile d'IA hybride couvre les trois dimensions fondamentales de l'expression musicale : rythme, mélodie et dynamique, au moyen de modèles spécialisés, légers et entraînables sans GPU.

2.9.2 Limitations identifiées

Les limites observées concernent surtout la sortie musicale brute des modèles. Elle reste exploitable, mais doit encore être encadrée pour garantir le registre instrumental, la stabilité harmonique et la lisibilité du rendu orchestral en conditions réelles.

2.9.3 Défis reportés vers la Construction

La phase de Construction devra donc concentrer l'effort sur le durcissement du produit : cohérence documentaire, correction des écarts mineurs de contrats entre services, notamment côté notation, et industrialisation des validations fonctionnelles et musicales.

2.10 Conclusion

L'Élaboration a transformé la vision architecturale en chaîne distribuée exécutable. Le pipeline SEDA produit des artefacts concrets à chaque étape, la pile de modèles hybrides gère les trois dimensions de l'expression musicale sans recourir à un GPU, et les diagrammes de séquence confirmés en pratique attestent de la solidité des contrats inter-services. La phase de Construction élargit désormais le périmètre vers la sécurité, la facturation, l'interface utilisateur et la qualité de production, en mettant en place un tableau de bord administrateur et en évaluant la performance globale du système de génération musicale en conditions réelles.

Chapitre 3

Construction : Services applicatifs, expérience utilisateur et qualité

3.1 Introduction

Un pipeline fonctionnel n'est pas encore une plateforme prête à l'emploi. La phase de Construction transforme ce pipeline en produit abouti, donnant à chaque composant la maturité nécessaire à un fonctionnement cohérent. Succédant à l'Élaboration centrée sur la validation du flux de génération et de l'orchestration, cette phase industrialise l'interface cliente, finalise la facturation, introduit un tableau de bord administrateur et consolide la qualité, la sécurité et l'observabilité du système.

3.2 Sécurité et authentification

La sécurité de la plateforme repose sur une authentification centralisée, des sessions validées côté serveur et un filtrage systématique des ressources appartenant à l'utilisateur courant. L'objectif est de protéger les comptes, les conversations et les artefacts générés sans exposer les services internes à des requêtes non autorisées.

3.2.1 Mécanismes principaux

Les sessions utilisent des jetons JWT stockés dans des cookies HttpOnly et Secure en production. Les mots de passe sont hachés avec bcrypt, tandis que les flux de vérification d'email et de réinitialisation s'appuient sur des jetons à durée de vie limitée. Les routes protégées vérifient la session avant toute action, puis filtrent les conversations, chansons, comptes et fichiers MIDI, WAV ou PDF selon l'identifiant du propriétaire.

Le contrôle est appliqué en couches. Le middleware intercepte les accès aux pages et aux routes privées, les handlers API revérifient l'identité avant d'exécuter une opération sensible.

Les réponses d'erreur restent neutres afin de ne pas révéler l'existence d'un compte. Cette répétition volontaire des vérifications limite les effets d'une mauvaise utilisation de l'interface cliente ou d'un appel direct à l'API.

La connexion Google OAuth complète le flux local sans contourner ces contrôles. Après validation du fournisseur, le serveur rattache l'identité à un compte interne et applique les mêmes quotas. Une limitation de débit, basée sur Redis avec repli en mémoire, protège les points d'entrée d'authentification. Les soumissions de génération sont en plus bloquées lorsque le solde de clés est insuffisant, ce qui évite de lancer un traitement coûteux pour une requête non éligible.

3.2.2 Flux d'authentification

Les diagrammes suivants résument les principaux cas d'usage côté client.

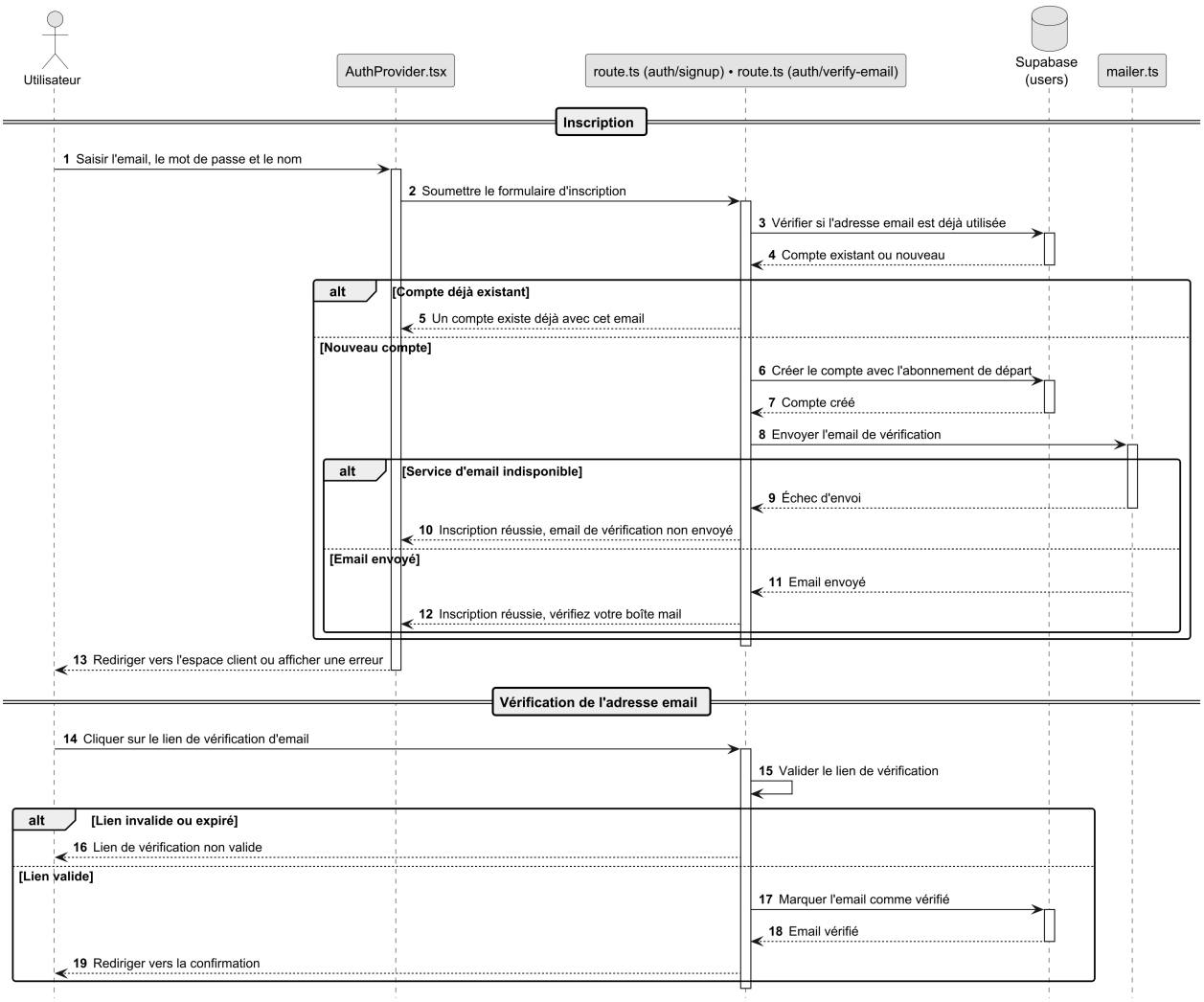


FIGURE 3.1 – Diagramme de séquence : processus d'inscription et de vérification d'adresse e-mail

L'inscription et la vérification d'email forment le socle d'activation du compte : création contrôlée, envoi d'un lien signé, puis validation serveur explicite avant confirmation de l'identité.

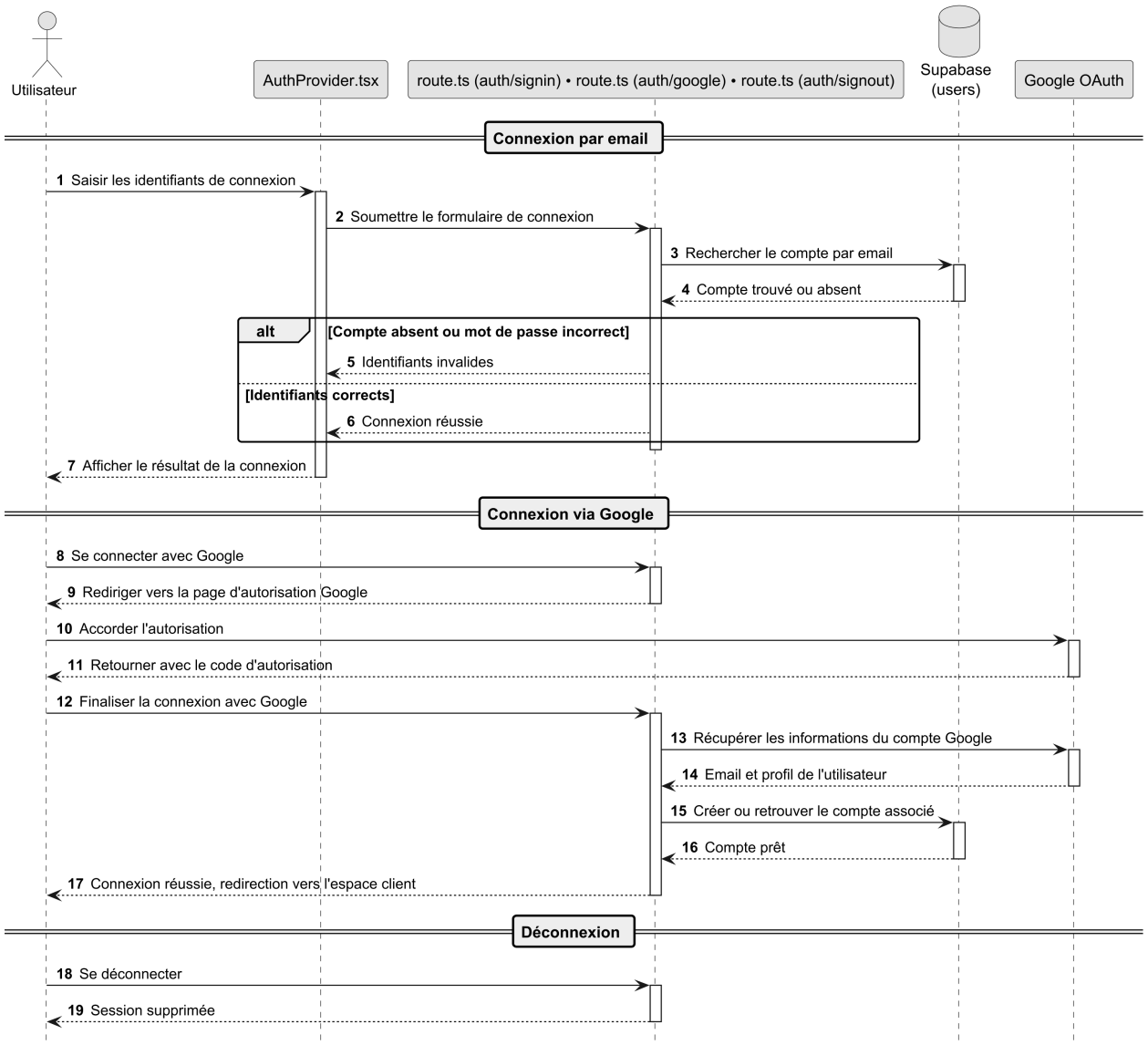


FIGURE 3.2 – Diagramme de séquence : processus de connexion et de déconnexion utilisateur

Les entrées de session (email ou Google) convergent vers le même service d'authentification, et la déconnexion ferme ce cycle par invalidation immédiate de la session active.

3.3 Facturation et gestion des bundles

3.3.1 Intégration Lemon Squeezy

Lemon Squeezy est utilisé comme Merchant of Record afin de déléguer entièrement le traitement des paiements, la gestion de la fiscalité (TVA comprise) et l'ensemble des obligations de conformité associées. L'application conserve uniquement les responsabilités métier : vérifier la session utilisateur, valider le bundle choisi, créer la session de paiement et créditer automatiquement les clés après confirmation du paiement.

Le flux reste court : le checkout est initié côté serveur, puis les webhooks signés synchronisent le paiement avec le portefeuille interne de l'utilisateur.

Le diagramme suivant résume l'initialisation du flux de facturation.

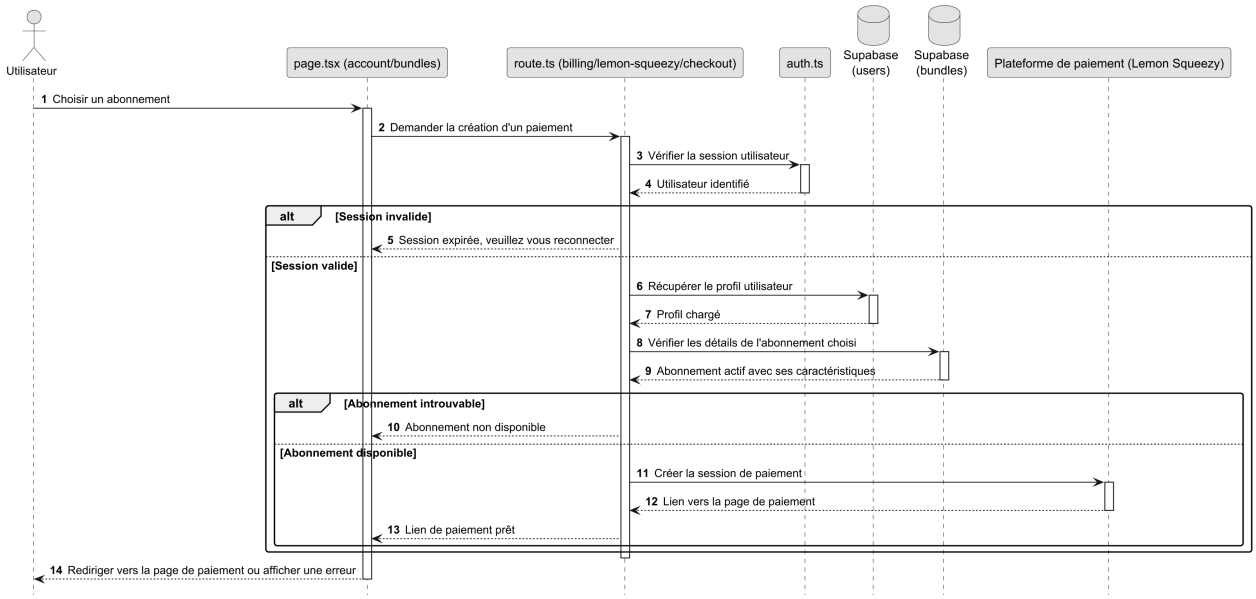


FIGURE 3.3 – Diagramme de séquence : facturation, création de la session de paiement (checkout)

Le serveur vérifie la session utilisateur et le bundle sélectionné avant toute redirection vers Lemon Squeezy. Le retour de paiement et le crédit automatique des clés sont détaillés dans l'annexe F.

Après le règlement, Lemon Squeezy transmet un webhook signé à l'API de la plateforme. Le serveur vérifie la signature HMAC, contrôle l'idempotence grâce à l'identifiant de l'événement, puis associe le paiement à l'utilisateur et au bundle acquis. Si l'événement est nouveau, les clés sont créditées sur le portefeuille et l'opération est consignée dans l'historique des achats.

3.3.2 Bundles de clés

Tous les utilisateurs partagent le même moteur de génération. La différenciation est strictement économique et se manifeste par le solde disponible, sans restriction fonctionnelle liée au palier choisi.

TABLEAU 3.1 – Bundles de clés

Bundle	Profil d'usage	Clés créditées	Usage cible
Prelude	Découverte et qualification	300	Qualification du pipeline et validation fonctionnelle
Concerto	Production régulière	5 000	Production régulière et montée en volume
Mozart	Usage avancé et intensif	15 000	Orchestrations larges et sessions intensives

3.3.3 Traitement des webhooks

L'implémentation repose sur trois garanties successives : validation HMAC de la signature, contrôle d'idempotence sur l'identifiant d'événement, puis enregistrement dans la table purchases. Ce contrôle en couches assure qu'une retransmission ou une défaillance réseau transitoire ne compromet ni la cohérence du solde ni la traçabilité comptable.

3.3.4 Limites d'utilisation et coût dynamique

Avant la mise en file, le serveur applique :

— vérification de l'éligibilité de l'utilisateur et du bundle courant,
— vérification des limites de session et du solde,
— calcul du coût dynamique à partir de deux termes : extraction d'intention Haiku + composition (couches, durée),
— déduction de crédits puis dispatch asynchrone vers `composeQueue`.

3.4 Application web cliente

L'application web cliente constitue l'interface principale de la plateforme, rendue en Server-Side Rendering (SSR). Elle se compose de deux espaces distincts : un espace public accessible sans authentification (présentation du produit, grille tarifaire, documentation et authentification), et un espace protégé en Client-Side Rendering (CSR) pour les interactions en temps réel, accessible après connexion, comprenant le studio conversationnel, le suivi des compositions et la gestion du compte.

3.4.1 Parcours public

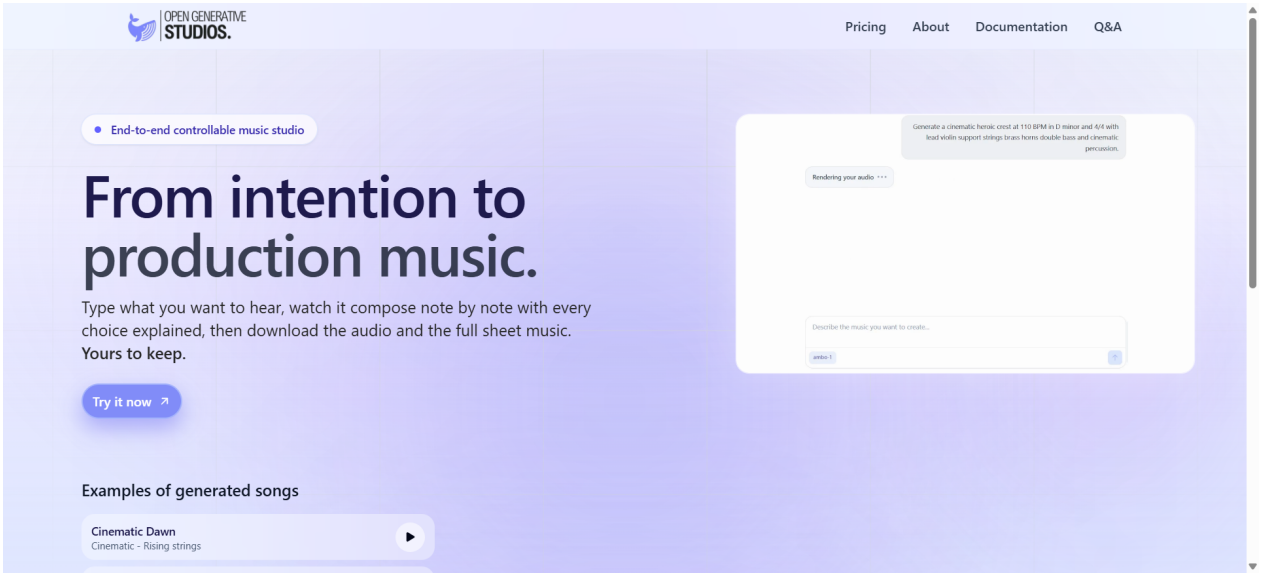


FIGURE 3.4 – Interface d'accueil de la plateforme.

Le visiteur découvre des exemples de musiques générées par le système ainsi qu'un jumbotron présentant le fonctionnement du chat et le parcours jusqu'au téléchargement de sa composition.

3.4.2 Authentification

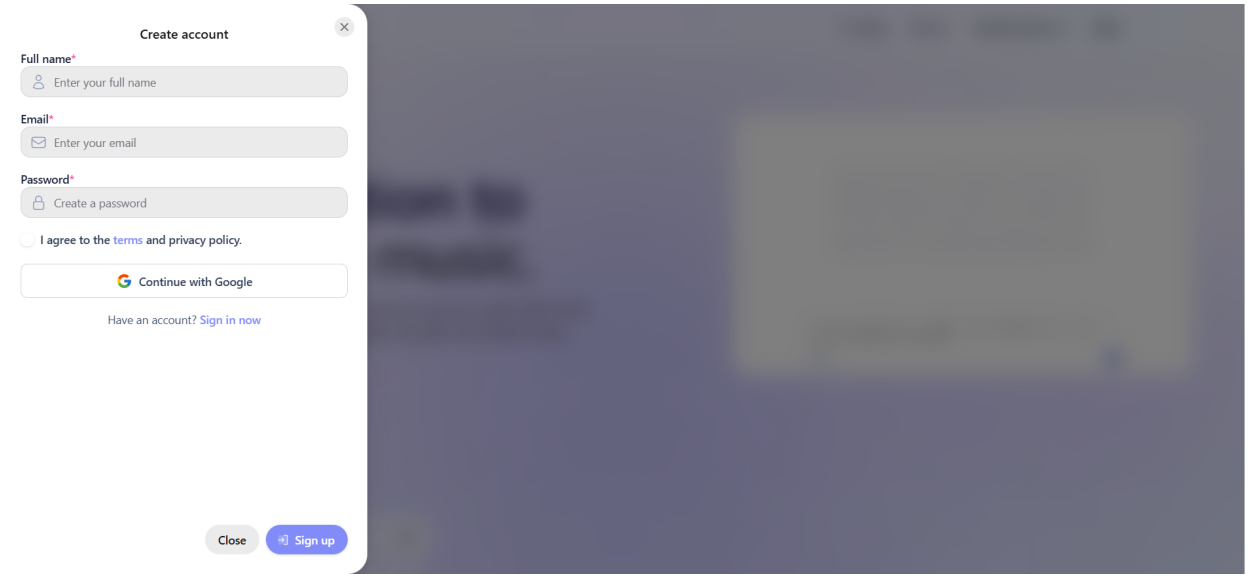


FIGURE 3.5 – Tiroir d'inscription et de connexion (email ou Google).

En cliquant sur « Try it now », un tiroir s'ouvre pour se connecter ou s'inscrire (email ou Google).

3.4.3 Studio conversationnel

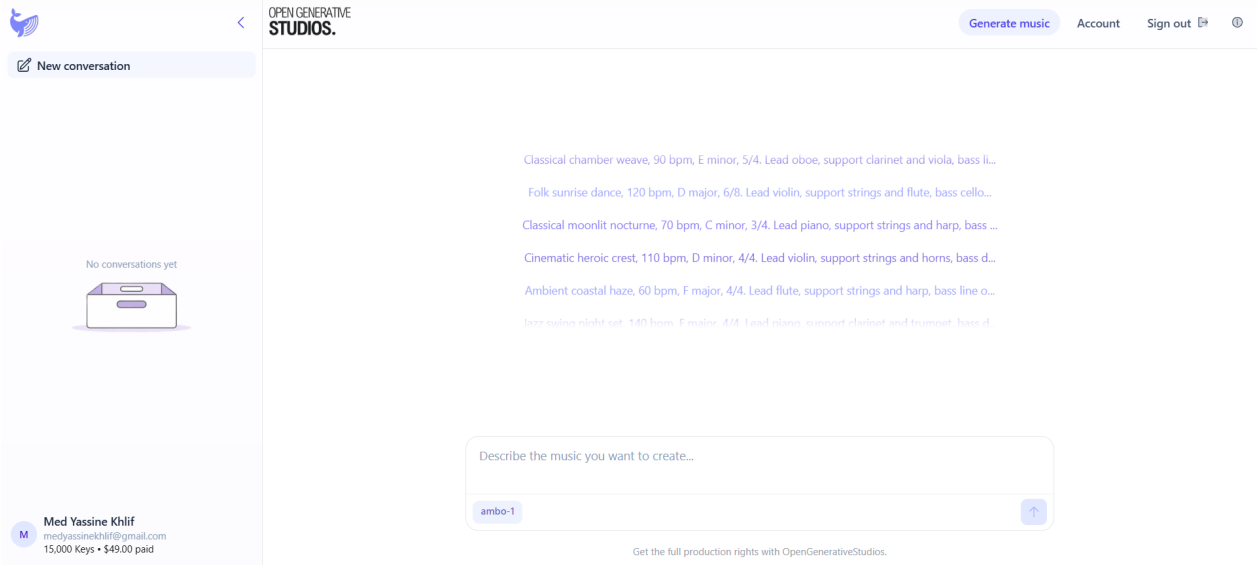


FIGURE 3.6 – Studio conversationnel : prompt, génération et téléchargement des artefacts.

L'utilisateur dispose d'une barre latérale lui permettant d'accéder à ses conversations. Des suggestions de prompts et de descriptions musicales s'affichent dynamiquement dans l'interface de chat.

3.4.4 Tableau de bord client

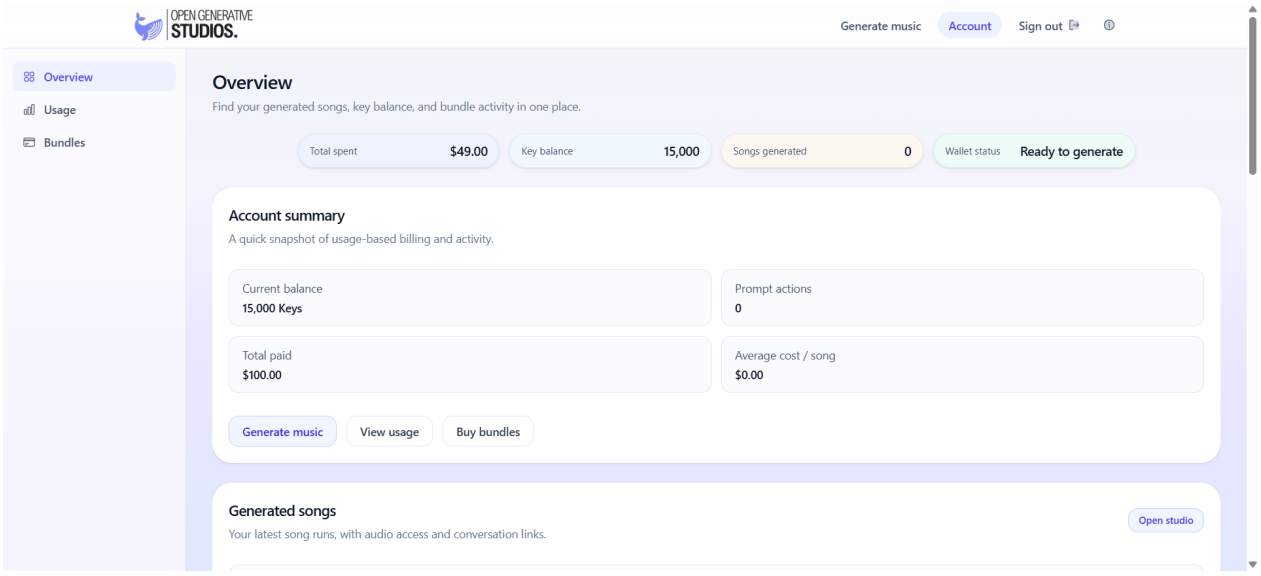


FIGURE 3.7 – Tableau de bord client : historique des compositions et artefacts.

L'utilisateur trouve sur cette page le montant total dépensé, son solde de clés, le nombre de morceaux générés ainsi qu'un récapitulatif des actions (prompts) avec le coût moyen par chanson.

3.4.5 Compte et bundles

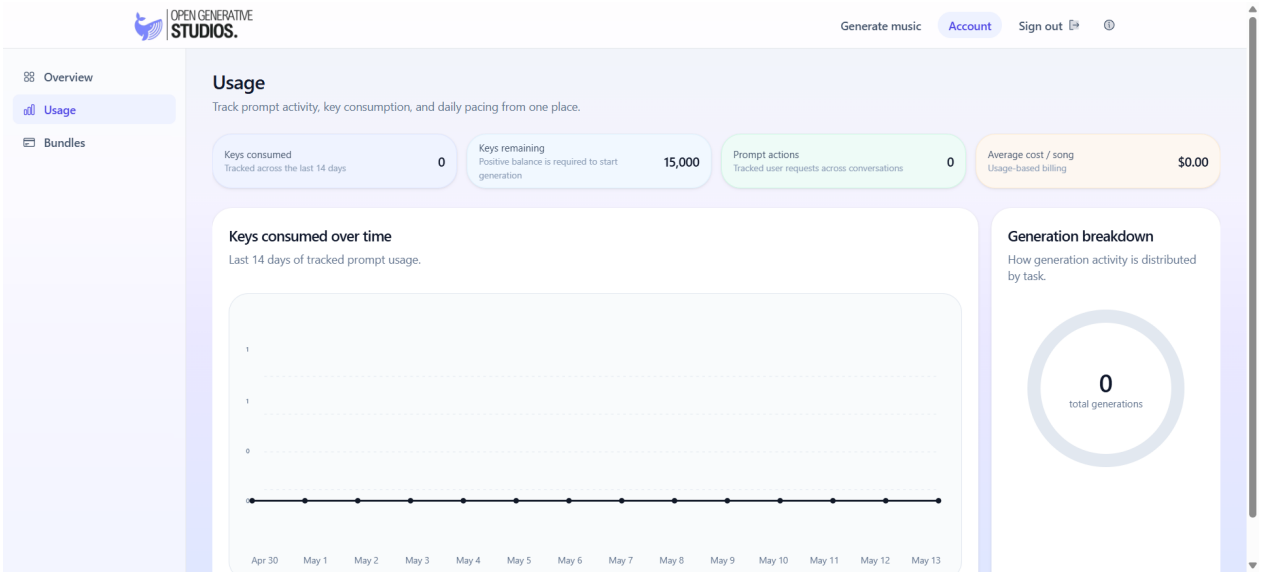


FIGURE 3.8 – Compte client : consommation de clés et gestion des bundles.

L'utilisateur dispose d'une vue synthétique du volume de clés consommées au fil du temps et du solde restant, ainsi que du coût moyen par morceau généré. Un graphique de répartition des activités par tâche (mélodie, harmonie, rythme, etc.) complète ce tableau de bord et offre un regard analytique sur la structure de ses compositions.

3.5 Tableau de bord administrateur

3.5.1 Objectif

Le tableau de bord administrateur gère les utilisateurs, les bundles et les coûts.

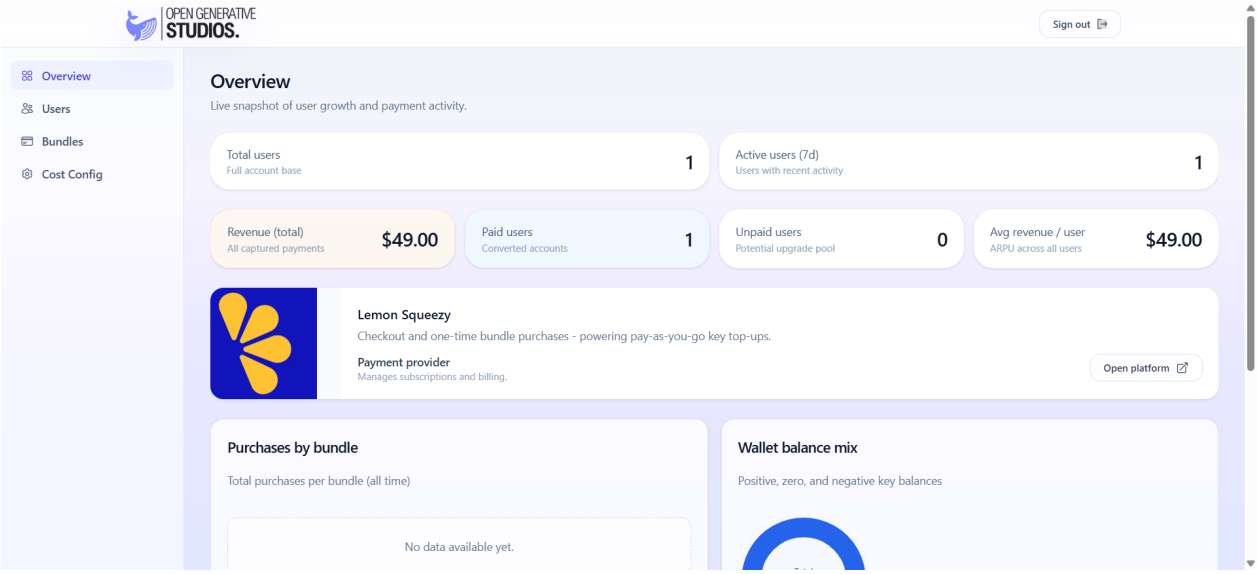


FIGURE 3.9 – Tableau de bord administrateur : indicateurs globaux, utilisateurs et revenus.

L'administrateur suit des indicateurs tels que les revenus afin d'orienter les décisions stratégiques.

3.5.2 Administration des utilisateurs et des bundles

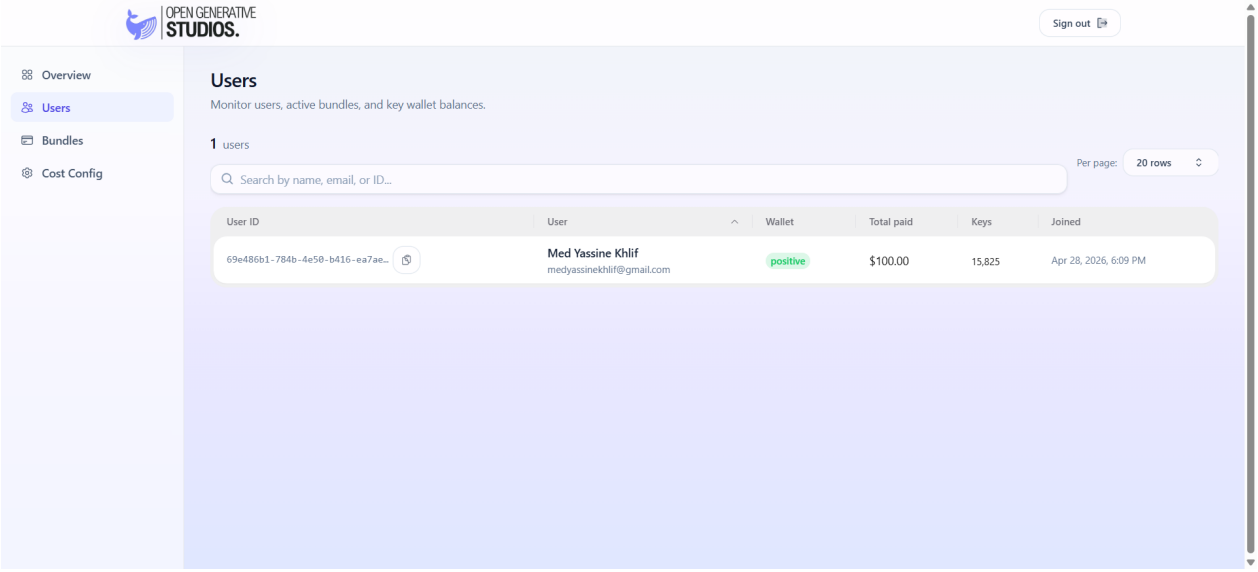


FIGURE 3.10 – Administration : liste des utilisateurs, soldes de clés et historique d'achat.

Une table paginée et segmentée permet à l'administrateur de parcourir les utilisateurs ligne par ligne.

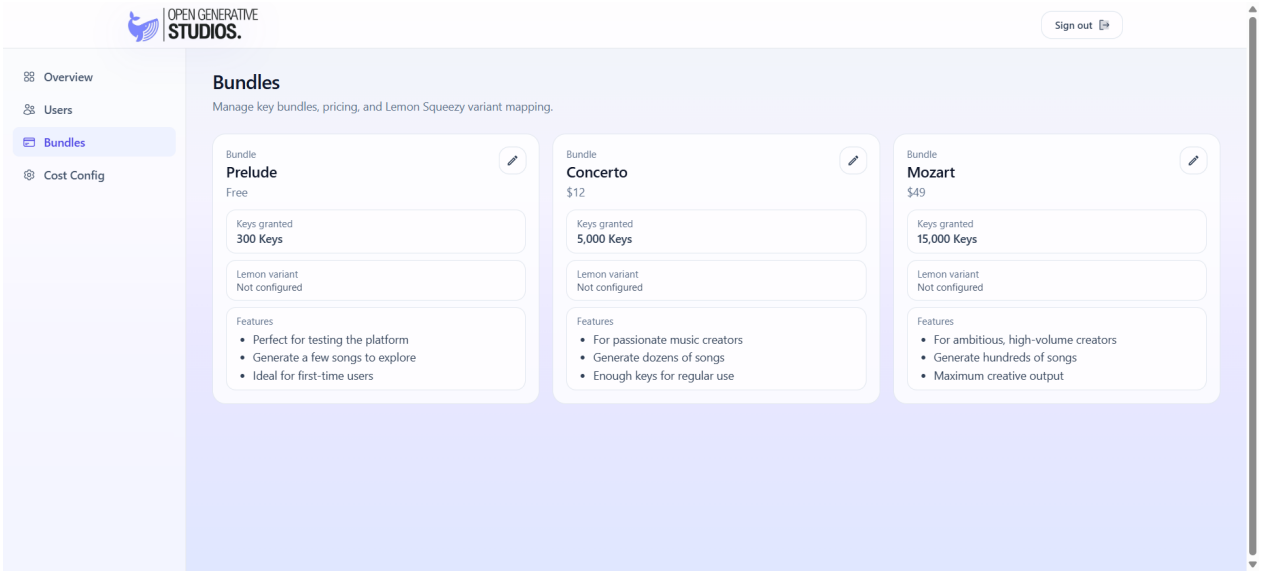


FIGURE 3.11 – Administration : configuration des bundles Prelude, Concerto et Mozart.

L'administrateur peut modifier à tout moment les prix des bundles ainsi que leurs descriptions.

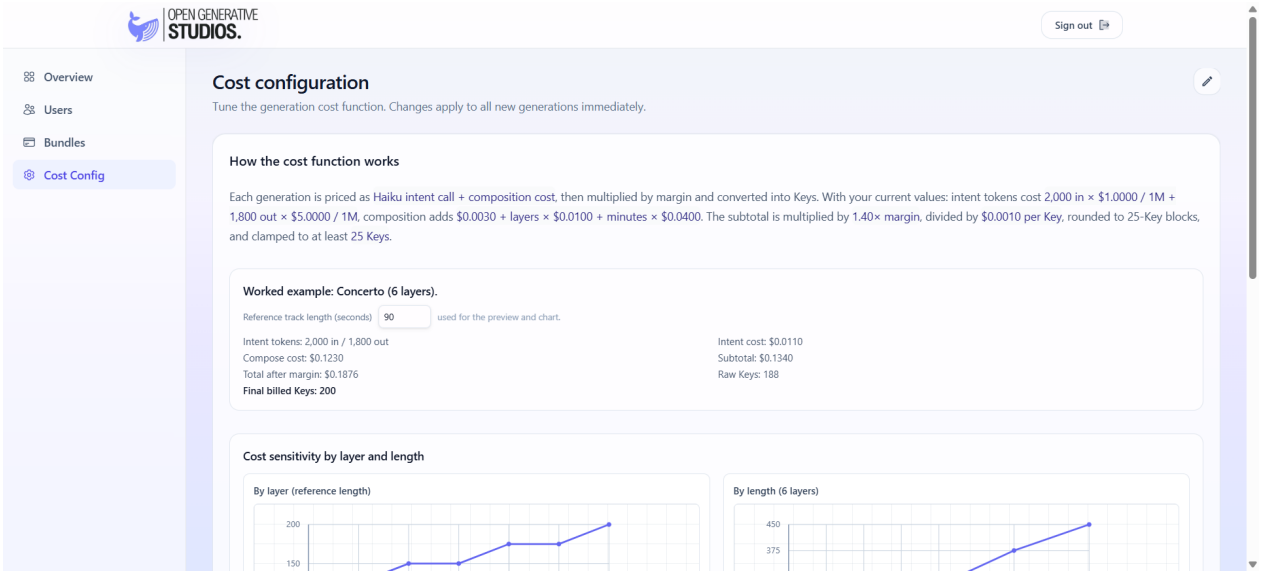


FIGURE 3.12 – Administration : configuration du coût de génération et tarification dynamique.

Il peut également mettre à jour le coût de génération selon la formule de coût définie.

3.5.3 Métriques et analytique

Le tableau de bord expose un ensemble de composants analytiques (cartes de métriques, graphiques d'évolution et indicateurs de performance) qui surveillent en permanence l'activité de la plateforme, son usage et ses données économiques. Les indicateurs retenus (consommation de clés, taux de génération, revenus et taux d'échec) croisent la dimension technique et la dimension économique, afin de piloter une plateforme en production.

Le diagramme de séquence suivant complète cette vue en détaillant deux parcours de gouvernance opérés depuis l'interface administrateur. Le premier concerne les bundles. L'administrateur ouvre la page des bundles, charge la liste depuis l'API, puis peut modifier un bundle existant, en créer un nouveau ou le supprimer. Le second porte sur la configuration des coûts. La page dédiée récupère les paramètres actuels, les présente à l'administrateur et transmet les nouvelles valeurs à l'API pour mise à jour. Dans les deux cas, l'interface n'accède pas directement à la base de données. Les lectures et écritures transitent par les routes administrateur avant d'être persistées dans Supabase.

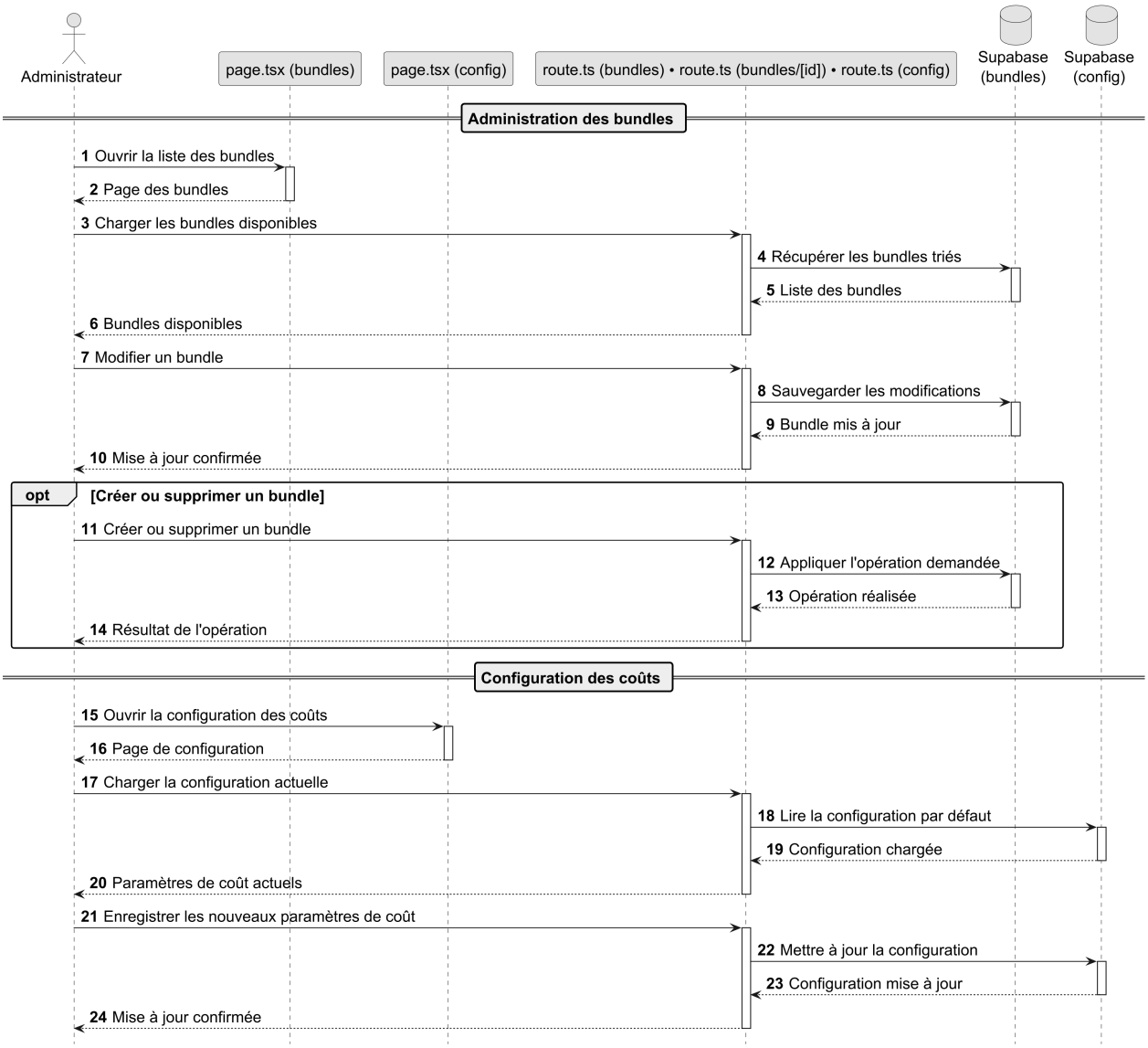


FIGURE 3.13 – Diagramme de séquence : administration, gestion des plans et configuration des coûts

Cette organisation centralise les décisions économiques sensibles en séparant deux responsabilités claires : la gestion des bundles et la configuration des coûts. Chaque modification transite obligatoirement par une couche API dédiée avant d'atteindre la base de données, ce qui garantit qu'aucun changement ne peut être appliqué sans validation intermédiaire. Ce point de contrôle unique assure la cohérence entre ce que l'interface affiche et ce que Supabase contient réellement.

3.6 Assurance qualité et observabilité

3.6.1 Positionnement opérationnel de la pile de composition

Le chapitre Élaboration présente les modèles, leurs données, leurs hyperparamètres et leurs métriques. Dans la phase Construction, l'objectif change profondément. Il ne s'agit plus d'exposer les résultats d'apprentissage, mais de vérifier ce que produisent ces modèles lorsqu'ils sont intégrés dans un parcours utilisateur complet. Les essais de génération ont confirmé que la pile probabiliste fournit une matière musicale exploitable, tout en révélant que cette matière demeure parfois trop libre pour garantir à elle seule une sortie orchestrale stable.

Concrètement, les modèles savent proposer des degrés, des attaques, des intensités et des durées. Ils ne garantissent pas toujours le bon registre instrumental, la consonance[M20] aux positions fortes, la séparation claire entre basse, harmonie et mélodie, ni la restitution cohérente des intentions musicales telles que certaines articulations[M21] ou effets rythmiques. La phase Construction a donc introduit une couche déterministe placée après la prédiction, dont le rôle est d'encadrer la sortie des modèles sans effacer leur diversité.

3.6.2 Conception de la couche déterministe

La couche déterministe a été conçue comme un ensemble de règles explicites activées après chaque génération. Certaines répondent à des défauts identifiés durant les tests, tels que des notes placées dans un registre peu naturel ou une mélodie atteignant une hauteur faible en début de mesure. D'autres ont été introduites par anticipation afin d'assurer la continuité du pipeline, notamment lorsqu'un artefact de modèle est absent ou lorsqu'une couche générée est vide.

a. Filtrage tonal et contrainte de registre

Le filtrage tonal constitue la décision la plus structurante. Une prédiction peut être statistiquement plausible tout en devenant problématique une fois projetée sur un instrument réel. Pour éviter ce cas, le système construit pour chaque instrument un ensemble de hauteurs autorisées à partir de la tonalité demandée et de la tessiture disponible dans le catalogue VPO, puis ramène les notes générées dans cet espace avant l'écriture MIDI. Ce filtrage intervient en aval du modèle de façon transparente : la logique de prédiction n'est pas modifiée, mais ses sorties sont corrigées dans le cas où elles déborderaient des limites définies.

Cette règle répond aux écarts entre validité symbolique et rendu orchestral. Une ligne trop grave, une doublure trop proche de la basse ou une note hors zone expressive peuvent fragiliser la couche, voire rendre le passage injouable. La contrainte de registre agit comme un garde-fou musical ancré dans les réalités physiques de chaque instrument : elle n'altère pas la prédiction, mais interdit les sorties contredisant ses limites, garantissant que chaque note reste dans un espace musicalement et techniquement valide.

b. Placement initial par rôle

Le placement initial complète ce filtrage. Les premières générations révélaient que deux couches distinctes pouvaient occuper la même zone sonore et perdre leur fonction dans le mix. Le moteur assigne désormais un point de départ cohérent avec le rôle de chaque couche. La basse est initialisée dans une zone grave, les harmonies occupent le registre médian et la mélodie démarre dans une zone plus lisible. Ce choix rend la séparation des couches plus stable dès les premières notes.

c. Alignement harmonique structurel

L'autre décision forte concerne les positions harmoniques. Les modèles peuvent produire une ligne intéressante localement tout en plaçant une note peu consonante sur un temps fort[M22] ou au début d'une mesure. L'alignement harmonique corrige ce point en rapprochant les hauteurs de la mélodie principale des tons d'accord[M23] de la progression en cours. La correction reste limitée aux positions structurelles afin de préserver le contour mélodique entre les mesures.

Ce mécanisme a été introduit pour traiter un problème audible. Une note faible placée au mauvais endroit produit une impression d'instabilité même lorsque le reste de la phrase est acceptable. En corrigeant uniquement les points d'appui, le système préserve la variabilité issue du modèle tout en renforçant l'assise harmonique du morceau.

d. Ajustement expressif post-prédiction

La composante d'expression fournit une base d'intensité et de durée, mais les essais ont montré que les valeurs brutes ne suffisent pas toujours à traduire l'évolution d'une section ou l'humeur demandée. Trois corrections sont appliquées après la prédiction.

- Le facteur d'énergie augmente ou réduit les intensités selon le niveau de la section.
- La nuance d'humeur introduit un décalage global lorsque l'intention requiert un rendu plus calme ou plus énergique.
- La plage dynamique du genre limite les valeurs extrêmes afin de conserver une identité sonore cohérente.

e. Modelage articulatoire et humanisation rythmique

Le modelage articulatoire et le swing[M24] constituent des réglages plus fins. Ils ne corrigent pas la structure principale du morceau, mais rendent les indications d'interprétation audibles. Le staccato[M25] raccourcit les notes en les détachant les unes des autres, le pizzicato[M26] accentue la coupure en simulant un jeu pincé, et le sustain[M27] prolonge légèrement la durée sonore pour adoucir les transitions. Ces trois modes agissent sur la durée des événements MIDI sans toucher aux hauteurs ni à la dynamique. Lorsque le style l'exige, le swing retarde certaines croches[M28] selon un ratio configurable, conférant un caractère plus souple et moins mécanique au phrasé.

f. Replis et robustesse

Les mécanismes de repli ont été intégrés de manière préventive afin d'assurer la continuité de la chaîne de génération malgré l'absence d'un modèle, d'un genre complet ou d'une couche fonctionnelle. En l'absence d'artefact mélodique, le moteur exploite les probabilités de pas propres au genre concerné. Si les motifs rythmiques sont indisponibles, une génération probabiliste simplifiée est utilisée. Lorsqu'aucun événement n'est produit, un fichier MIDI valide mais silencieux est néanmoins généré. Cette dégradation contrôlée renforce la robustesse du système, préserve les services aval et facilite l'identification des anomalies.

3.6.3 Profils d'efficacité des composantes

L'audit confirme la compatibilité de la pile hybride avec un déploiement sans GPU. La composante d'expression (~249 Mo) reste viable sur CPU grâce à la vectorisation par section. Les composantes mélodique et rythmique présentent une empreinte mémoire négligeable avec une inférence quasi instantanée. La chaîne déterministe post-prédiction présente une complexité linéaire sans surcoût d'apprentissage. L'ensemble valide l'éviction de tout modèle autorégressif lourd, dont la latence serait incompatible avec un rendu quasi temps réel sur infrastructure mutualisée.

3.6.4 Validation au niveau des services

La logique critique de chaque service a été vérifiée individuellement, couvrant l'exactitude de la génération de MIDI brut, la fiabilité du post-traitement MIDI, le comportement du rendu audio et la lisibilité de la notation en sortie. Les scénarios de repli relatifs à une couche vide, un instrument absent du catalogue ou un genre sans modèle ont été explicitement couverts.

3.6.5 Vérification de l'intégration entre les étapes de files d'attente

La validation de l'intégration a porté sur la chaîne complète de files d'attente Redis :

$$\texttt{composeQueue} \rightarrow \texttt{midiQueue} \rightarrow \texttt{renderQueue} \rightarrow \texttt{notationQueue}.$$

Pour chaque transition, le contrat de payload a été vérifié au regard de la présence et du format des champs requis, de la cohérence du chemin d'artefact transmis entre services, et de la transmission correcte des métadonnées d'intention incluant genre, tempo et instrumentation, afin de s'assurer qu'aucune transition ne repose sur des hypothèses implicites concernant la structure des données échangées.

Les scénarios d'échec inter-services ont également été couverts, notamment l'arrêt brutal d'un service consommateur, la reprise après redémarrage et la détection des messages orphelins en file d'attente morte (DLQ), validant que le système réagit de manière prévisible sans perte silencieuse de messages ni blocage du pipeline.

3.6.6 Tests de scénarios de bout en bout

Des prompts représentatifs couvrant différents genres, configurations instrumentales et durées ont été exécutés de la soumission jusqu'à la disponibilité finale des artefacts WAV, MIDI et PDF. Ces tests ont permis de valider la cohérence des métadonnées d'intention avec les fichiers produits et de détecter les cas où les replis automatiques étaient déclenchés.

3.6.7 Surveillance opérationnelle

Les endpoints de santé, les vérifications de profondeur de file d'attente, le suivi du taux d'échec et l'analyse de la file d'attente morte ont été utilisés pour surveiller la qualité d'exécution. La passerelle de notification centralise les événements started, completed et failed de chaque service.

3.7 Bilan de la Construction

3.7.1 Résultats atteints

Au terme de cette phase, le système couvre l'intégralité du chemin produit, de la création de compte jusqu'aux artefacts orchestraux téléchargeables. La sécurité, la facturation, l'observabilité et le diagnostic ont été construits comme des composantes de première importance, et non ajoutés en périphérie. Sur le plan musical, les tests en conditions réelles ont confirmé que la couche déterministe et les profils d'efficacité produisent des sorties stables et conformes aux intentions musicales formulées.

3.7.2 Limitations identifiées

Les acquis fonctionnels sont confirmés, mais la validation s'est intégralement déroulée en environnement local. Les comportements propres à un environnement de production, tels que la charge réseau réelle, la latence inter-services ou les conditions de démarrage à froid, n'ont pas encore pu être observés ni mesurés.

3.7.3 Défis reportés vers la Transition

Certains sujets ont délibérément été mis en attente pour ne pas surcharger cette phase. La Transition s'en chargera : déploiement multi-cloud en environnement de production, mise en place du pipeline de livraison continue, configuration du domaine et de l'infrastructure de sécurité réseau, validation de bout en bout du parcours utilisateur, et bilan des apprentissages méthodologiques accumulés tout au long du projet. Ces points sont traités en phase de clôture pour leur accorder l'attention qu'ils méritent.

3.8 Conclusion

La phase de Construction a tenu ses engagements : ce qui n'était qu'un pipeline fonctionnel est devenu une plateforme que l'on peut réellement exploiter, surveiller et sécuriser au quotidien, où la sécurité multi-niveaux, la facturation par bundles, les interfaces studio et administrateur, et le durcissement déterministe de la génération forment un ensemble solide et cohérent dont la maturité se rapproche des standards d'une mise en production. La phase de Transition fermera la boucle : déploiement multi-cloud, infrastructure de production et retour réflexif sur ce que ce projet aura véritablement appris.

Chapitre 4

Transition : Déploiement, exploitation et perspectives

4.1 Introduction

La phase de Transition ne vise pas à enrichir le système, mais à démontrer que ce qui a été construit est déployable, reproductible et conforme aux exigences d'un projet réel. C'est ici que les décisions architecturales prises en amont sont confrontées à la réalité d'un environnement de production : la livraison du système ne se postule plus, elle se prouve. Cette phase consolide les acquis techniques à travers un pipeline de déploiement stabilisé et une validation de bout en bout du parcours utilisateur, ancrant ainsi le système dans les contraintes concrètes d'un produit mis en production.

4.2 Infrastructure et déploiement

4.2.1 Environnement local

Docker Compose permet de démarrer l'intégralité du système en une seule commande. L'environnement est identique d'une machine à l'autre, les problèmes de dépendances sont éliminés, et le développeur travaille sur une base stable et auditée.

4.2.2 Architecture de production

Le déploiement cible est multi-cloud, en tirant parti des atouts de chaque fournisseur. Leur choix ne repose pas uniquement sur des critères techniques. La maturité de l'écosystème a également été déterminante : des années d'utilisation en production, une documentation de référence et des communautés actives capables d'apporter des réponses de fond constituent des facteurs réduisant l'exposition aux risques opérationnels et améliorant la capacité de réponse face aux incidents.

La figure ci-dessous représente la topologie de déploiement en production.

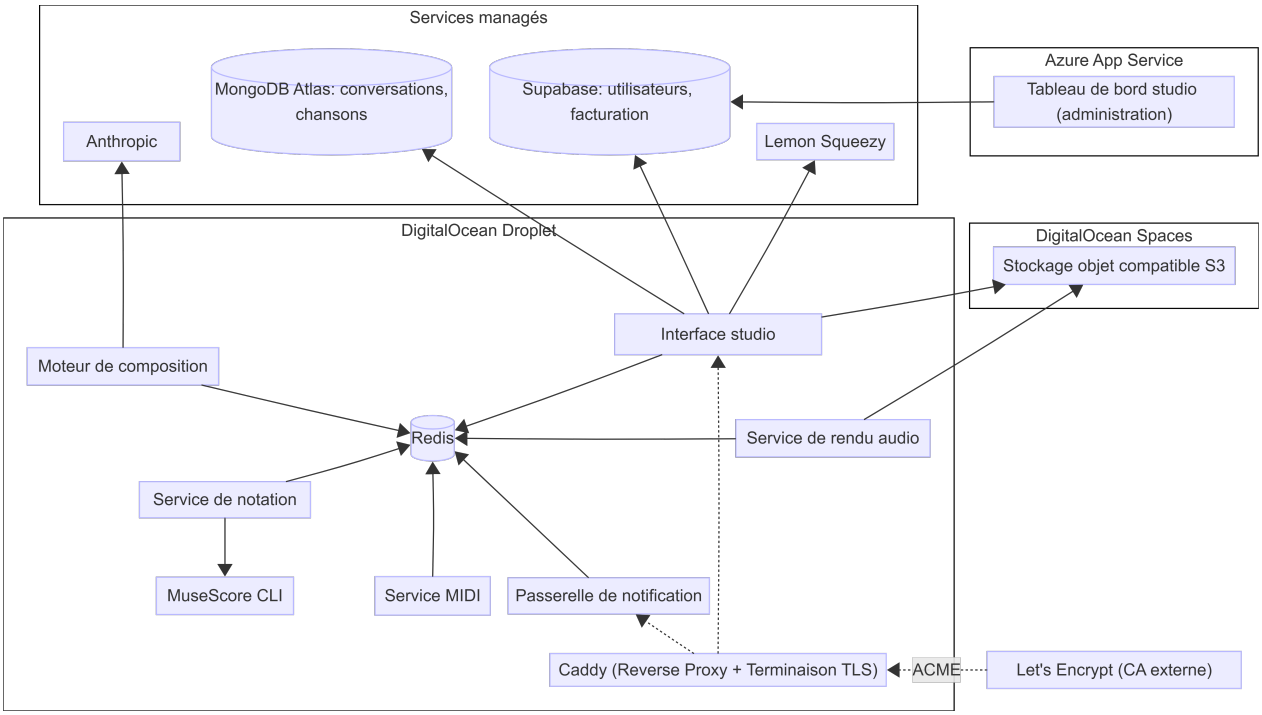


FIGURE 4.1 – Architecture de déploiement multi-cloud en production

— **DigitalOcean Droplet** accueille le cœur du système : l'application web, les cinq services du pipeline et Redis. Ce regroupement réduit la latence inter-services et garantit un déploiement homogène de toute la chaîne de calcul. La machine cible est un Droplet Ubuntu 24.04 LTS en région FRA1 (Francfort), doté de 4 Go de RAM, 2 vCPU et 80 Go de SSD, une configuration suffisante pour absorber la charge des inférences en temps réel.

  Chaque service étant encapsulé dans un conteneur Docker indépendant et communiquant via des files Redis, aucune dépendance n'existe entre eux au niveau du code. Cette séparation garantit que la consolidation sur un seul Droplet est un choix d'infrastructure propre au stade actuel du projet : lors du passage à l'échelle, chaque service pourra être déployé sur un Droplet dédié avec des ressources adaptées à son profil de charge, sans modifier le code applicatif.

— **Azure App Service** héberge le tableau de bord administrateur, entièrement découplé du Droplet : il communique directement avec Supabase et MongoDB Atlas, et reste opérationnel même en cas de défaillance du pipeline de génération. Cette isolation garantit que la supervision et la gestion des comptes ne sont jamais compromises par un incident sur les workers.

— **DigitalOcean Spaces** assure le stockage des artefacts générés. La compatibilité S3, la diffusion CDN et les URLs signées garantissent une distribution sécurisée des fichiers WAV, MIDI et PDF. Le Droplet et le bucket étant tous deux hébergés en région FRA1, les transferts s'effectuent sans traverser de frontière réseau, minimisant la latence et éliminant les coûts de bande passante sortante.

— **Supabase** et **MongoDB Atlas** opèrent comme services de données managés, assurant haute disponibilité, sauvegardes automatiques et observabilité sans opération manuelle. Supabase, construit sur PostgreSQL, bénéficie d'une communauté open source active, tandis que MongoDB Atlas est la référence des bases documentaires managées pour les projets à schéma flexible.

4.2.3 Terminaison TLS et reverse proxy

Caddy assure la terminaison TLS et le reverse proxy pour l'ensemble du trafic entrant. Il expose deux points d'entrée : le domaine principal acheminé vers l'application web sur le port 3000, et le sous-domaine de notification acheminé vers la passerelle sur le port 8088.

La gestion des certificats repose sur le protocole ACME et l'autorité de certification Let's Encrypt. Dès le démarrage, Caddy négocie automatiquement un certificat TLS valide auprès de Let's Encrypt, le stocke localement et planifie son renouvellement bien avant expiration, sans aucune intervention manuelle. Cette automatisation élimine le risque de certificat expiré, qui constitue l'une des causes les plus fréquentes d'interruption de service non planifiée. L'échange ACME s'effectue via un défi HTTP-01 : Let's Encrypt sollicite un jeton sur un chemin prédéfini du domaine, Caddy répond, et la validité du contrôle du domaine est établie. Toute la chaîne, provisionnement, stockage et renouvellement, est transparente pour le reste du système.

4.2.4 Domaine et configuration DNS

Le domaine de la plateforme a été enregistré auprès de **name.com**. Une fois le DNS propagé et les certificats provisionnés par Caddy, la plateforme est accessible à l'adresse suivante :

opengenerativestudios.studio

Deux enregistrements A ont été créés pour faire pointer l'apex du domaine et le sous-domaine de notification vers l'adresse IP du Droplet, permettant à Caddy de résoudre les défis ACME et d'acheminer le trafic dès la propagation DNS.

TABLEAU 4.1 – Enregistrements DNS principaux

Type	Hôte	Valeur	TTL
A	@	IP du Droplet	300
A	ws	IP du Droplet	300

Le TTL de 300 secondes (5 minutes) a été retenu délibérément : il réduit le temps de propagation lors de la mise en service initiale et accélère les éventuelles corrections d'adresse, au prix d'un trafic de résolution DNS légèrement supérieur, sans conséquence à cette échelle.

4.2.5 Infrastructure de messagerie transactionnelle

La messagerie transactionnelle est assurée par **Resend**, qui route les envois via **Amazon SES**. Trois flux d'expédition distincts sont configurés, chacun associé à une adresse dédiée :

— **support@opengenerativestudios.studio** : communications générales et notifications de facturation
— **verify@opengenerativestudios.studio** : vérifications d'adresse à la création de compte
— **security@opengenerativestudios.studio** : réinitialisations de mot de passe et alertes de sécurité

Cette séparation en flux indépendants répond à plusieurs exigences de production. Elle assure une **isolation de délivrabilité** : une dégradation de réputation sur le flux de vérification, provoquée par exemple par un pic d'inscriptions, n'affecte pas les flux critiques de facturation ou de sécurité. Elle renforce la **confiance de l'utilisateur** par un signal contextuel clair : l'adresse d'expédition correspond à l'usage attendu, ce qui réduit les signalements abusifs en spam. Enfin, elle permet un suivi analytique et opérationnel par catégorie, avec des templates distincts, des limites de débit, des journaux et des alertes configurables indépendamment pour chaque flux.

L'authentification des envois repose sur trois enregistrements DNS configurés sur name.com et vérifiés par Resend :

— **DKIM** : signature cryptographique des en-têtes de message, permettant aux serveurs destinataires de vérifier que les e-mails n'ont pas été altérés en transit et qu'ils émanent bien d'un expéditeur autorisé par le domaine.
— **SPF** : enregistrement TXT déclarant les serveurs autorisés à envoyer au nom du domaine, ce qui réduit le risque d'usurpation d'identité et améliore le score de réputation auprès des filtres anti-spam.
— **DMARC** : politique d'alignement qui indique aux serveurs destinataires comment traiter les messages échouant aux contrôles DKIM et SPF, et permet la collecte de rapports d'abus exploitables pour la supervision.

Ces trois mécanismes combinés constituent le socle standard de délivrabilité exigé par les principaux fournisseurs de messagerie. Ils couvrent l'intégralité du cycle utilisateur : création de compte, vérification d'adresse, réinitialisation de mot de passe et notifications de facturation.

4.2.6 Sécurité et déploiement continu

La terminaison TLS, l'exposition minimale du pare-feu et l'isolation des secrets par environnement constituent des contrôles obligatoires. Les variables sensibles sont injectées par environnement et ne transitent jamais dans les images Docker.

Un pipeline CI/CD GitHub Actions automatise le lint, les tests et la livraison continue vers les environnements cibles, garantissant que toute modification du code est validée et livrée de façon systématique.

4.2.7 Analyse des coûts

Le tableau ci-dessous recense l'ensemble des composants du système, leur fournisseur, leur avantage principal et le coût mensuel associé. Les coûts fixes s'élèvent à 29 $/mois. Les coûts variables, liés à l'usage de l'API Anthropic et aux commissions Lemon Squeezy, sont proportionnels au volume de générations et de transactions.

TABLEAU 4.2 – Composantes du système, fournisseurs et coûts

Fournisseur	**Avantage clé**	**Coût**
DigitalOcean Droplet	Latence interne faible, runtime unifié	24 $/mois
Azure App Service F1	Publication web et supervision intégrées	Gratuit
DigitalOcean Spaces	Stockage objet, CDN FRA1, URLs signées	5 $/mois
Supabase (PostgreSQL)	SQL managé, authentification et accès par rôle	Gratuit
MongoDB Atlas	Scalabilité documentaire, haute disponibilité native	Gratuit
name.com	Enregistrement, DNS et hébergement e-mail	Gratuit
Resend	3 000 e-mails par mois inclus, routage via Amazon SES	Gratuit
GitHub Actions	Lint, tests et livraison automatisée	Gratuit
Anthropic API	Conversion du prompt en intention musicale structurée	Variable
Lemon Squeezy	Gestion des commandes, webhooks et fiscalité	Variable
	Total fixe mensuel	**29 $**

Ce déploiement a été réalisé dans le cadre du GitHub Student Developer Pack. Ce programme couvre une part significative des coûts fixes : DigitalOcean offre un crédit de 200 $ valable un an, name.com fournit un nom de domaine gratuit pour la première année, et Azure met à disposition son plan App Service F1 accompagné d'un crédit de 100 $ pour les étudiants. À hauteur de 29 $/mois pour les coûts fixes, l'infrastructure est opérationnelle sans coût réel pendant six à huit mois, couvrant intégralement la durée du projet. Les coûts variables liés à l'API Anthropic et à Lemon Squeezy sont proportionnels à l'usage réel : l'API est facturée à la génération et Lemon Squeezy prélève une commission par transaction, sans abonnement fixe.

4.3 Leçons apprises

4.3.1 Décisions architecturales

Le choix de SEDA comme modèle d'exécution s'est révélé structurant [3]. L'isolation des composants et le confinement des pannes ont tenu leurs promesses, au prix de contrats de messages stricts et d'un versionnage discipliné des payloads, une rigueur qui est la condition même de la robustesse du système.

La décision de préférer une pile de modèles légers et spécialisés à un unique modèle auto-régressif a été pleinement validée. Chaque composante est remplaçable indépendamment, l'ensemble s'exécute sans GPU, et les améliorations peuvent cibler une dimension précise sans perturber le reste du pipeline.

4.3.2 Trajectoire d'expérimentation Machine Learning

L'architecture retenue est le fruit d'un parcours d'expérimentation, fondé sur une exploration méthodique de l'espace de conception et non sur un choix immédiat.

La première piste, fondée sur trois modèles de Random Forest couvrant respectivement la mélodie, le rythme et l'expression, se heurtait à une limite structurelle : ce type de modèle ne capture pas explicitement les dépendances temporelles et la continuité inhérentes à la musicalité d'une phrase. D'autres familles d'algorithmes ont ensuite été évaluées, introduisant chacune de nouvelles contraintes en termes de coût computationnel, de contrôle stylistique via le langage naturel, et de cohérence avec la structure musicale sous-jacente. Aucune ne satisfaisait simultanément les exigences de légèreté, de modularité, de reproductibilité et de respect de la structure musicale, sans recours au GPU.

Ces contraintes ont progressivement orienté la conception vers une approche hybride combinant des modèles spécialisés pour chaque composante musicale.

4.3.3 Apprentissages en systèmes distribués

Quatre principes se sont avérés aussi déterminants que la qualité algorithmique des modèles.

L'observabilité ne peut pas être différée.

L'absence initiale de la passerelle de notification rendait tout diagnostic aveugle : chaque service consignait ses événements isolément et l'analyse reposait sur une collecte manuelle de logs. La passerelle a agrégé les notifications et ajouté un identifiant de corrélation par exécution, rendant le parcours d'un job lisible dans un flux unique. Dès sa mise en service, la durée moyenne de diagnostic des incidents a été divisée par quatre.

L'idempotence des traitements est non négociable.

Lorsqu'un service redémarre en cours d'exécution, le mécanisme BRPOP repêche le message non acquitté et relance le traitement [16]. Sans idempotence, un traitement relancé peut générer des artefacts dupliqués ou corrompus. Une vérification d'état Redis préalable en début de traitement a suffi à éliminer ce risque dans tous les cas.

Les blocages silencieux de file constituent le mode de défaillance le plus dangereux.

Une `renderQueue` saturée n'est pas visible par l'utilisateur : le job reste à l'état **queued** indéfiniment. Des sondes de profondeur de file sur l'endpoint /jobs, couplées à une alerte sur dépassement de seuil, ont rendu ces situations détectables avant qu'elles ne s'accumulent.

La DLQ est un outil de diagnostic, non un silo d'échecs.

Structurer les payloads d'erreur avec l'identifiant de la chanson, l'étape d'échec, le message d'exception et l'horodatage a permis de corréler les défaillances entre services. Il est ainsi apparu que 80 % des échecs du service MIDI provenaient d'intentions malformées issues du moteur de composition, et non de défauts internes au service. Sans ce niveau de détail, les corrections auraient été appliquées au mauvais endroit.

4.4 Bilan de la Transition

4.4.1 Résultats atteints

La validation finale confirme des exécutions réussies de bout en bout, du prompt utilisateur à la génération des artefacts WAV et PDF, sous une orchestration de production. Le système est reproductible en Compose local comme dans la disposition multi-cloud cible. La plateforme expose un parcours utilisateur, de la création de compte jusqu'aux artefacts orchestraux téléchargeables, avec sécurité, facturation et observabilité intégrées.

4.4.2 Limitations identifiées

Deux limitations délimitent le périmètre de validité du système.

La première concerne la qualité sonore. Le pipeline repose sur des bibliothèques SFZ, retenues pour leur compatibilité avec une exécution headless sans DAW [20, 21]. Ces bibliothèques restituent fidèlement les familles orchestrales, les tessitures et les dynamiques, et remplissent pleinement leur rôle de validation. Le passage à des bibliothèques de meilleure qualité supposerait une intégration DAW d'une ingénierie délicate, tant au niveau de la gestion des tampons audio que du déploiement, car les greffons lourds et le rendu en temps réel mobilisent des ressources significatives côté serveur.

La seconde concerne la couverture musicale. Les modèles ont été entraînés sur MAESTRO et Lakh MIDI, deux corpus centrés sur la musique occidentale. D'autres genres, tels que les musiques orientales, arabes ou andalouses, avec leurs gammes modales et leurs structures rythmiques asymétriques, n'y sont pas représentés et constituent une direction naturelle pour les travaux ultérieurs.

4.4.3 Perspectives

Plusieurs directions d'évolution se dessinent naturellement à partir des limites identifiées.

La plus ambitieuse à moyen terme concerne l'intégration des musiques orientales et arabes, absentes des corpus actuels malgré la richesse de leurs gammes modales et de leurs structures rythmiques, et dont la prise en compte ouvrirait le système à une diversité musicale aujourd'hui inaccessible.

Sur le plan des modèles, les Random Forests pourraient être complétés par des approches de boosting comme XGBoost ou LightGBM, qui produisent généralement de meilleurs résultats sans exiger davantage de ressources, et ces différents modèles pourraient être combinés via une approche de stacking afin de tirer parti de leurs complémentarités respectives. Ce qui paraît le plus prometteur à moyen terme, c'est de connecter la génération aux retours réels des utilisateurs : notes, préférences, écoutes répétées. Un ajustement incrémental sur cette base ferait évoluer le système bien au-delà de ce qu'un entraînement statique peut offrir.

4.5 Conclusion

La phase de Transition confirme la maturité du livrable. La plateforme est déployée, documentée et reproductible. L'ensemble du cycle du Processus Unifié a produit, à chacune de ses phases, des artefacts concrets et vérifiables.

Ce bilan atteste que les choix architecturaux ont tenu leurs engagements en production, que les apprentissages tirés du développement ont enrichi la compréhension du domaine, et que les limitations identifiées délimitent avec précision l'espace des améliorations futures. Les perspectives dégagées prolongent naturellement la trajectoire du système sans en remettre en cause les fondations, ce qui reste le critère le plus fiable de la qualité d'une conception logicielle.

Conclusion générale

« La musique est une révélation plus haute que toute sagesse et toute philosophie. »

Ludwig van Beethoven

Beethoven ne voyait pas la musique comme un simple art parmi d'autres. Pour lui, elle touchait à une vérité que ni la raison ni les mots ne pouvaient atteindre seuls : une émotion que l'on ressent avant même de la comprendre. C'est précisément cette conviction qui anime "Open Generative Studios". Non pas remplacer l'émotion musicale par la logique d'une machine, mais offrir à cette émotion un chemin nouveau, tracé par le génie logiciel, afin qu'elle reste accessible à tous.

Créer de la musique a toujours été un acte profondément humain. Ce projet repose sur un pari audacieux : montrer que la discipline du génie logiciel, loin d'aseptiser cette magie, peut au contraire la rendre plus lisible, plus partageable et plus vivante. Là où d'autres systèmes dissimulent leurs rouages derrière des interfaces opaques, ce projet érige la transparence en principe fondateur. Chaque transformation, du mot tapé par l'utilisateur jusqu'à la note qui résonne, est traçable, compréhensible et justifiable.

Ce pari a été tenu. Un utilisateur peut aujourd'hui exprimer une intention musicale en langage naturel et voir naître, en retour, une œuvre qui lui ressemble. Sans matériel coûteux, sans expertise préalable, sans mystère inaccessible. La machine apprend, s'adapte et prolonge la création humaine. Surtout, elle explique : chaque choix qu'elle pose peut être suivi, compris et remis en question. Dans un domaine où l'opacité est trop souvent la norme, cette approche constitue déjà une forme de révolution tranquille.

Mais ce qui a été construit ne représente qu'une première phrase d'une longue histoire. Les prochaines versions se rêvent collaboratives, capables d'entendre le monde avec plus de finesse et de traduire l'émotion avec plus de profondeur [31, 32]. Chaque limitation d'aujourd'hui est une invitation lancée à demain. L'objectif reste inchangé depuis la première ligne de code : offrir à quiconque, compositeur aguerri ou simple curieux, les clés d'un studio génératif qui lui ressemble vraiment.

Parce qu'au fond, la meilleure partition est celle que l'on n'a pas encore écrite…

Annexe A : Référentiel technologique et audio

TABLEAU A.1 – Technologies utilisées

Technologie	Définition
NEXT.JS	**Next.js (App Router)** : framework React full-stack pour le rendu serveur, le routage et les API.
TS	**TypeScript** : sur-ensemble typé de JavaScript ajoutant un système de types statiques et l'outillage associé.
	React : bibliothèque JavaScript pour construire des interfaces utilisateur déclaratives basées sur des composants.
	Tailwind CSS : framework CSS utilitaire qui permet de composer le style via des classes atomiques.
hero	**HeroUI** : bibliothèque de composants UI pour React.
	Node.js : runtime JavaScript côté serveur basé sur V8.
	Redis : base de données en mémoire clé-valeur pour cache, files et pub/sub.
	Python : langage de programmation généraliste pour scripts, data et backend.
	MongoDB : base de données NoSQL orientée documents.
	Supabase (PostgreSQL) : plateforme backend open source construite sur PostgreSQL, offrant base de données, authentification et stockage.
	DigitalOcean (Droplets et Spaces) : fournisseur cloud proposant des VM et du stockage objet compatible S3.
	Claude Haiku : modèle de langage d'Anthropic orienté vers des réponses rapides et légères.
	Scikit-learn : bibliothèque Python de machine learning classique (régression, classification, clustering).
	NumPy : bibliothèque Python pour le calcul scientifique et l'algèbre linéaire.

kaggle	**Kaggle** : plateforme de data science pour notebooks, compétitions et jeux de données.
	**Hugging Face** : plateforme de partage de modèles et jeux de données ML, avec hub et bibliothèques.
	**mido** : bibliothèque Python pour lire, écrire et manipuler des fichiers MIDI.
sfz	**SFZ / Sfizz** : SFZ est un format ouvert de description d'instruments échantillonnés, tandis que Sfizz est un lecteur SFZ open source.
	**Pedalboard** : bibliothèque Python pour chaînes d'effets audio (EQ, compression, réverbération).
	**MuseScore CLI** : interface en ligne de commande pour convertir et exporter des partitions.
	**JWT** : standard JSON Web Token pour représenter et signer des jetons d'authentification.
	**Google OAuth** : implémentation OAuth 2.0 pour l'authentification déléguée.
	**Docker** : plateforme de conteneurisation pour packager et exécuter des applications de manière isolée. **Docker Compose** : outil d'orchestration multi-conteneurs basé sur des fichiers YAML.
	**GitHub Actions** : service CI/CD intégré à GitHub pour automatiser des workflows.
	**Caddy** : serveur web et reverse proxy avec gestion TLS automatique.
	**Let's Encrypt** : autorité de certification gratuite et automatisée fournissant des certificats TLS.
	**Name.com** : registrar de domaines et fournisseur de services DNS.
	**Azure App Service** : plateforme PaaS pour héberger des applications web.
	**Lemon Squeezy** : plateforme de paiement et Merchant of Record pour produits numériques.
Resend	**Resend** : service d'envoi d'emails transactionnels.
LaTeX	**LaTeX** : système de composition typographique pour documents scientifiques.
	**Grammarly** : assistant de correction grammaticale et stylistique.
	**Draw.io** : outil de création de diagrammes et schémas.
	**Mermaid** : langage de diagrammes textuels pour générer des graphes.

Annexe B : Catalogue VPO

Le VPO (Virtual Playing Orchestra) est une bibliothèque d'instruments orchestraux échantillonnés, distribuée au format SFZ et lisible via un lecteur SFZ. Elle permet de restituer un rendu orchestral à partir de données MIDI dans un DAW. Dans ce projet, le catalogue VPO sert de référentiel pour valider les instruments demandés, leur tessiture et leurs mappings, afin d'assurer une compatibilité directe avec le rendu audio. Le visuel ci-dessous montre un orchestre et illustre le fait que les instruments et patchs VPO sont orientés vers les pupitres orchestraux.



FIGURE B.1 – Orchestre (visuel d'illustration du VPO)

Annexe C : Dictionnaire musical détaillé

Tableau C.1 – Dictionnaire des termes musicaux

Nº	Terme	Définition
1	**Couche MIDI (layer)**	Piste ou couche symbolique contenant les notes d'un instrument ou d'un rôle.
2	**Dynamique**	Variation d'intensité perçue (piano, forte), souvent pilotée par la vélocité MIDI. En clair, jouer plus doux ou plus fort.
3	**Section (intro, outro)**	Segment formel d'une pièce (intro, couplet, refrain, outro) qui organise le déroulement temporel. C'est l'équivalent d'un chapitre dans une histoire musicale.
4	**Famille orchestrale (cordes, vents, cuivres, percussions)**	Catégorie d'instruments partageant un timbre et une fonction, par exemple les cordes, vents, cuivres et percussions. Cela aide à répartir les rôles sonores.
5	**Vélocité**	Valeur MIDI (0–127) associée à l'attaque, qui contrôle l'intensité et parfois le timbre. Plus la valeur est haute, plus la note est forte.
6	**Tempo**	Vitesse d'exécution en battements par minute (BPM). Par exemple, 60 BPM signifie un battement par seconde.
7	**Tonalité**	Organisation des notes autour d'une tonique et d'un mode (majeur ou mineur). C'est le "centre tonal", par exemple do majeur.
8	**Tessiture**	Étendue de notes qu'un instrument peut jouer confortablement sans tension. En dehors, le son devient difficile ou peu naturel.
9	**Phrasé**	Manière d'articuler et de lier les notes pour former des phrases musicales. C'est comparable à la ponctuation et à l'intonation d'une phrase parlée.
10	**Registre (grave, médium, aigu)**	Zone de hauteur relative dans laquelle se situe un son ou une partie. Grave = bas, aigu = haut.
11	**Hauteur (pitch)**	Variation d'une note selon sa fréquence et sa valeur MIDI : plus la fréquence est élevée, plus elle est aiguë.

12	**Intervalle**	Distance de hauteur entre deux notes, mesurée en demi-tons ou en degrés. C'est le "saut" entre deux notes.
13	**Classe de hauteur / Chroma**	Hauteur modulo l'octave. 12 classes (do, do#, ré, etc.). Deux notes séparées d'une octave partagent la même classe.
14	**Octave**	Intervalle de 12 demi-tons, correspondant à un doublement de fréquence. Monter d'une octave, c'est la même note plus aiguë.
15	**Attaque (onset)**	Instant de début d'une note (attaque), utilisé pour l'analyse rythmique. Il indique précisément quand la note commence.
16	**Degré (de gamme)**	Position d'une note dans la gamme par rapport à la tonique (I à VII). Utile pour décrire les notes sans donner leur nom absolu.
17	**Gamme diatonique**	Gamme de 7 notes (majeure ou mineure) issue du système tonal. C'est la base des musiques occidentales courantes.
18	**Groove**	Profil rythmique récurrent et sensation de placement qui donne la pulsation. C'est le "balancement" qui fait bouger.
19	**Mesure**	Unité rythmique délimitée par la signature (barre). Elle regroupe un nombre fixe de battements.
20	**Consonance**	Qualité d'un intervalle ou d'un accord perçue comme stable et reposante. Elle s'oppose aux tensions de la dissonance.
21	**Articulation**	Manière d'attaquer et de lier les notes (lié, détaché, accent), qui influence le caractère du jeu.
22	**Temps fort**	Battement accentué d'une mesure, point d'appui rythmique où l'on attend la stabilité harmonique.
23	**Accord**	Ensemble de plusieurs notes jouées simultanément formant une entité harmonique.
24	**Swing**	Déplacement rythmique qui retarde une partie des subdivisions (souvent les croches) pour créer un balancement.
25	**Staccato**	Articulation détachée où les notes sont raccourcies pour produire un rendu sec.
26	**Pizzicato**	Technique de cordes consistant à pincer les cordes au lieu de les frotter, donnant un timbre percussif.
27	**Sustain**	Tenue prolongée d'une note, souvent en allongeant sa durée ou sa résonance.
28	**Croche**	Valeur rythmique équivalente à la moitié d'une noire, souvent écrite en groupes.

Annexe D : Exemple de partition : Ode à la joie de Beethoven

Une partition musicale est le document de référence d'une œuvre, consignant les notes, rythmes, nuances, tempos et articulations nécessaires à son interprétation. La partition ci-dessous présente l'Ode à la joie de Beethoven, thème emblématique du quatrième mouvement de sa Neuvième Symphonie.



FIGURE D.1 – Exemple de partition : Ode à la joie de Beethoven

Annexe E : Rôle et structure du document d'intention musicale

Le fichier d'intention est l'artefact central produit par le moteur de composition avant tout traitement MIDI. Il formalise l'analyse du prompt utilisateur en décisions musicales cohérentes et validées.

Il contient les champs suivants :

- **Thème** : humeurs, genre primaire, genre secondaire et son poids de mélange, artistes de référence.
- **Forme** : sections (nom, mesures, bin énergie/densité), tempo en BPM, tonalité, signature, mode et feel, plus modulations lorsque activées.
- **Instruments** : liste demandée (nom, rôle, priorité), évitements, articulation, indications de registre, surcharges par section (niveau de vélocité), plus résolution catalogue avec le nom résolu, score de similarité et plage MIDI min/max.
- **Mixage** : réglages par instrument (volume en décibels, panoramique, réverbération courte et longue, filtre passe-haut en hertz) et bus globaux (taux de mouillé des bus court et long, crête maximale en décibels).
- **Contraintes** : nombre maximal de couches, durée cible en secondes et politique de mise en cache (priorité à la vitesse ou priorité à la qualité).

Le validateur enrichit l'intent en résolvant les instruments via le catalogue VPO, ajoute les plages MIDI et émet des avertissements en cas de substitutions. Lorsque certaines dépendances essentielles ne peuvent pas être résolues, le processus de composition peut être interrompu. Chaque worker consomme ensuite ce document pour piloter sa génération, ce qui rend chaque étape autonome et rejouable.

Annexe F : Confirmation de paiement et crédit des clés via webhook

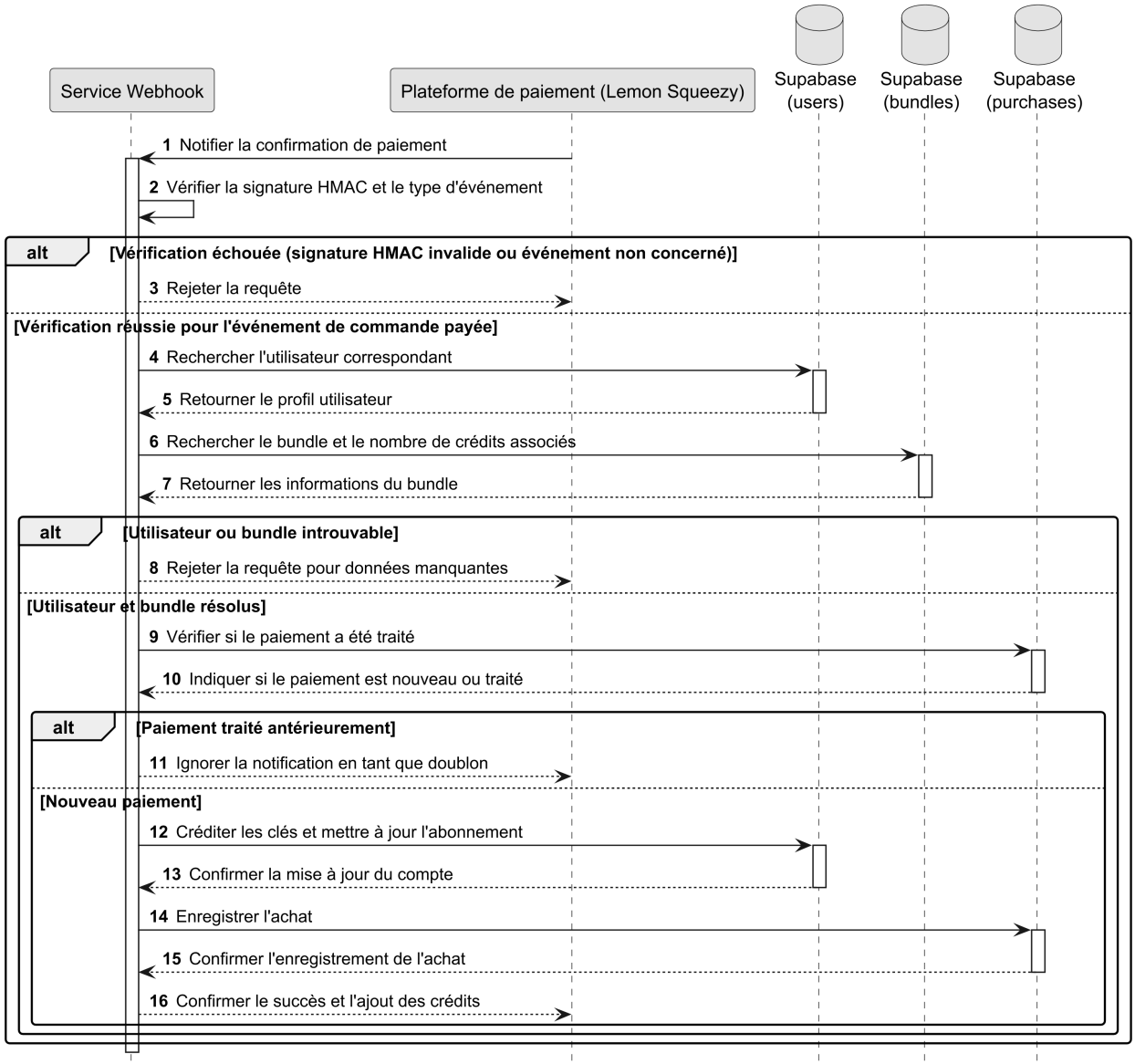


FIGURE F.1 – Diagramme de séquence : facturation, confirmation et crédit des clés via webhook

Après paiement, le webhook vérifie la signature HMAC et le type d'événement, résout l'utilisateur et le bundle, contrôle l'idempotence, puis crédite les clés. Signatures invalides, données manquantes et doublons ne modifient jamais le portefeuille.

Annexe G : Référence des endpoints API

Tableau G.1 – Endpoints API client et administrateur

Endpoint	Description
Client	
POST /api/auth/signin	Connexion email/mot de passe
POST /api/auth/signup	Création de compte
POST /api/auth/signout	Déconnexion et suppression de session
GET /api/auth/me	Profil utilisateur courant
GET /api/auth/verify-email	Validation du lien de vérification
POST /api/auth/resend-verification	Renvoi d'un email de vérification
POST /api/auth/forgot-password	Initiation de réinitialisation
POST /api/auth/reset-password	Finalisation de réinitialisation
GET /api/auth/google	Redirection vers Google OAuth
GET /api/auth/google/callback	Callback OAuth et ouverture de session
GET /api/account/analytics	Statistiques d'utilisation du compte
GET /api/account/bundles	Bundles disponibles pour l'achat de clés
GET /api/account/purchases	Historique des achats
GET /api/account/songs	Liste des morceaux générés
POST /api/billing/lemon-squeezy/checkout	Création de session de paiement
POST /api/billing/lemon-squeezy/webhook	Réception des webhooks Lemon Squeezy
GET /api/conversations	Liste des conversations
POST /api/conversations	Création d'une conversation
GET /api/conversations/[id]	Détails d'une conversation
PUT /api/conversations/[id]	Renommage d'une conversation
DELETE /api/conversations/[id]	Suppression d'une conversation
POST /api/conversations/[id]/messages	Ajout d'un message et lancement de génération
PATCH /api/conversations/[id]/messages	Interruption d'une génération
DELETE /api/conversations/[id]/messages	Retrait d'un tour de conversation
GET /api/songs/[songId]	Métadonnées, statut et disponibilité d'un morceau
PATCH /api/songs/[songId]	Demande d'édition d'un morceau
GET /api/songs/[songId]/audio	Téléchargement du fichier WAV
GET /api/songs/[songId]/sheet	Téléchargement de la partition PDF
POST /api/songs/[songId]/sheet	Mise en file de l'export PDF
Administrateur	
POST /api/auth/sign-in	Connexion au tableau de bord administrateur
POST /api/auth/sign-out	Déconnexion du tableau de bord administrateur
GET /api/config	Lecture de la configuration de coût et de génération
PATCH /api/config	Mise à jour de la configuration de coût et de génération
GET /api/users	Liste paginée et filtrable des utilisateurs
PATCH /api/users/[id]	Mise à jour du solde de clés d'un utilisateur
GET /api/bundles	Liste des bundles de clés configurés
POST /api/bundles	Création d'un bundle de clés
PATCH /api/bundles/[id]	Mise à jour d'un bundle de clés
DELETE /api/bundles/[id]	Suppression d'un bundle de clés

Bibliographie

[1] The Business Research Company. **Artificial Intelligence (AI) in Music Market Report 2026**. 2026. https://www.thebusinessresearchcompany.com/report/artificial-intelligence-ai-in-music-global-market-report

[2] I. Jacobson, G. Booch, J. Rumbaugh. **The Unified Software Development Process**. Addison-Wesley, 1999.

[3] M. Welsh, D. Culler, E. Brewer. **SEDA : An Architecture for Well-Conditioned, Scalable Internet Services**. In *Proceedings of SOSP 2001*. ACM, 2001.

[4] Suno Inc. **Introducing v4.5**. 2025. https://suno.com/blog/introducing-v4-5

[5] J. Righteous. **Suno AI : Master Metatags & Create Music Your Way**. Updated 2026. https://jackrighteous.com/blogs/guides-using-suno-ai-music-creation/understanding-suno-ai-a-comprehensive-guide-to-metatags

[6] Musicful AI. **Suno vs Udio : Which AI Song Generator Delivers Better Music ?** 2025. https://www.musicful.ai/vs/suno-vs-udio/

[7] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi, M. Sharifi, N. Zeghidour, C. Frank. **MusicLM : Generating Music From Text**. arXiv :2301.11325, 2023. https://arxiv.org/abs/2301.11325

[8] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, A. Défossez. **Simple and Controllable Music Generation**. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://arxiv.org/abs/2306.05284

[9] Meta AI / Replicate. **meta/musicgen : Readme and Docs**. 2023. https://replicate.com/meta/musicgen/readme

[10] AudioCipher. **How to Use Meta's MusicGen for Music Production Workflows**. 2025. https://www.audiocipher.com/post/meta-musicgen

[11] CometAPI. **Best 3 AI Music Generation Models of 2025**. 2026. https://www.cometapi.com/best-3-ai-music-generation-models-of-2025/

[12] Siteefy. **AIVA – Features, Pricing, Pros & Cons**. 2026. https://siteefy.com/tools/aiva

[13] AI Flow Review. **AIVA Review (2025) : My Hands-On Test, Full Feature Analysis**. 2025. https://aiflowreview.com/aiva-review-2025/

[14] SOUNDRAW. **SOUNDRAW : AI Music Generator**. 2020. https://soundraw.io/

[15] Boomy Corporation. **Boomy : Make Music with Artificial Intelligence**. 2021. https://boomy.com/

[16] Redis. **Redis Lists**. Consulté en mars 2026. https://redis.io/docs/latest/develop/data-types/lists/

[17] Anthropic. **Claude : AI Assistant by Anthropic**. 2023. https://www.anthropic.com/claude

[18] F. Pedregosa, G. Varoquaux, A. Gramfort et al. **Scikit-learn : Machine Learning in Python**. *Journal of Machine Learning Research*, 12 :2825–2830, 2011.

[19] MIDI Manufacturers Association. **MIDI 1.0 Detailed Specification**, version 4.2.1. 1996. https://www.midi.org/specifications

[20] P. Ferrand et al. **sfizz : A SFZ Player Library and LV2 Plugin**. 2024. https://sfizz.com/

[21] SFZ Format Specification. **SFZ v2 Reference**. 2019. https://sfzformat.com/

[22] Spotify. **Pedalboard : A Python Library for Audio Effects**. 2021. https://github.com/spotify/pedalboard

[23] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, D. Eck. **Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset**. In *International Conference on Learning Representations (ICLR 2019)*. https://magenta.tensorflow.org/datasets/maestro

[24] C. Raffel. **Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching**. PhD Thesis, Columbia University, 2016. (Lakh MIDI Dataset) https://colinraffel.com/projects/lmd/

[25] J. Gillick, A. Roberts, J. Engel, D. Eck, D. Bamman. **Learning to Groove with Inverse Sequence Transformations**. In *International Conference on Machine Learning (ICML 2019)*. https://magenta.tensorflow.org/datasets/groove

[26] F. Foscarin, A. McLeod, P. Rigaux, F. Jacquemard, M. Sakai. **ASAP : A Dataset of Aligned Scores and Performances for Piano Transcription**. In *ISMIR 2020*. https://github.com/fosfrancesco/asap-dataset

[27] L. Breiman. **Random Forests**. *Machine Learning*, 45(1) :5–32, 2001.

[28] D. Huron. **Sweet Anticipation : Music and the Psychology of Expectation**. MIT Press, 2006.

[29] J. A. Hartigan, M. A. Wong. **Algorithm AS 136 : A K-Means Clustering Algorithm**. *Journal of the Royal Statistical Society, Series C*, 28(1) :100–108, 1979.

[30] WHATWG. **HTML Living Standard : Server-sent events**. Consulté en mars 2026. https://html.spec.whatwg.org/multipage/server-sent-events.html

[31] G. Nierhaus. **Algorithmic Composition : Paradigms of Automated Music Generation**. Springer-Verlag Wien, 2009.

[32] C. Roads. **The Computer Music Tutorial**. MIT Press, 1996.